

Machine Learning

CMPSCI 589

Bruno C. da Silva

bsilva@cs.umass.edu

Neural Networks (pt. 2)

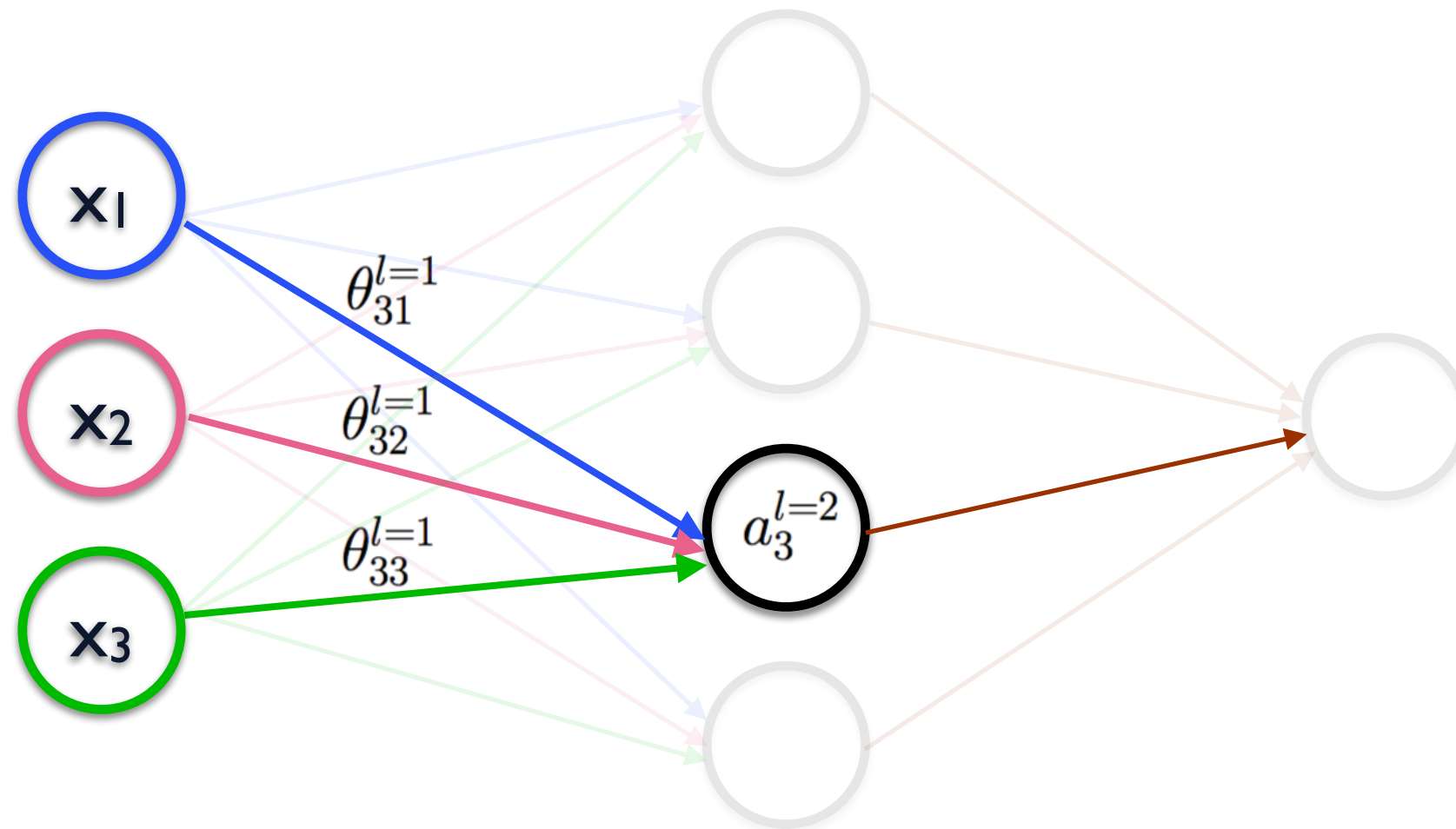
UMassAmherst
College of Information
& Computer Sciences



This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/)



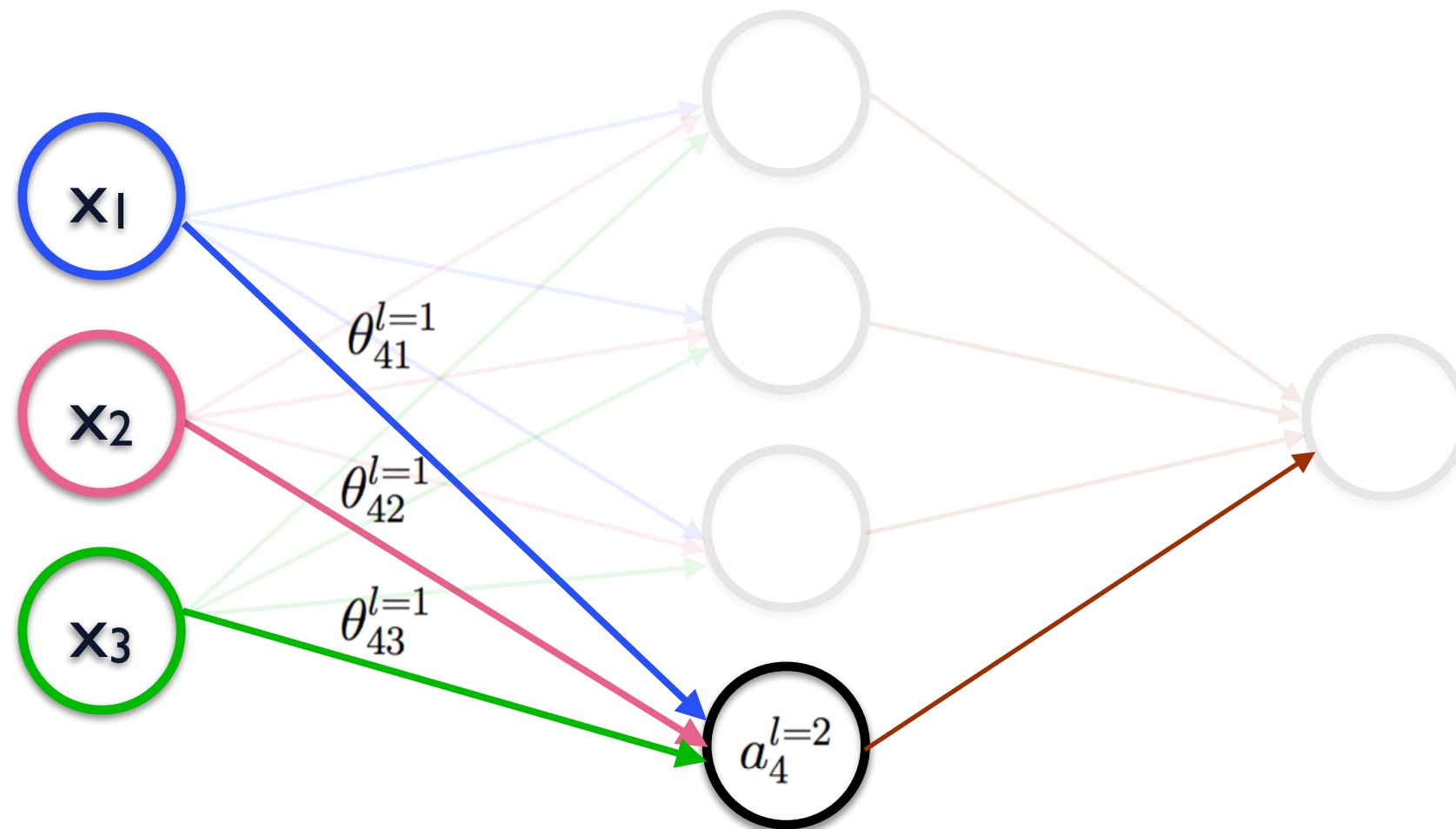
Review: Neural Networks



Computing the output/activation of a single neuron

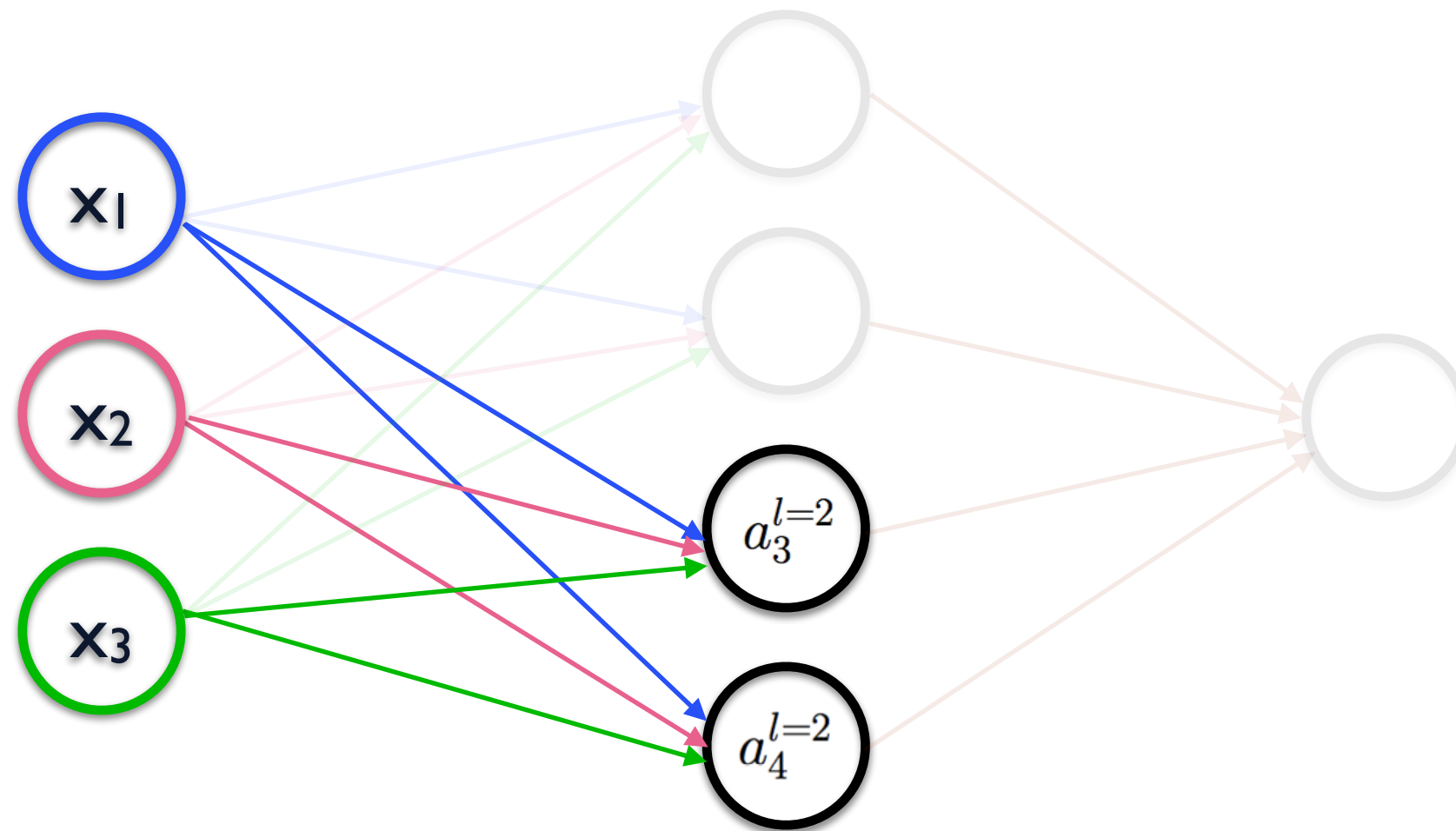
$$a_3^{l=2} = g\left(x_1\theta_{31}^{l=1} + x_2\theta_{32}^{l=1} + x_3\theta_{33}^{l=1}\right)$$

Review: Neural Networks



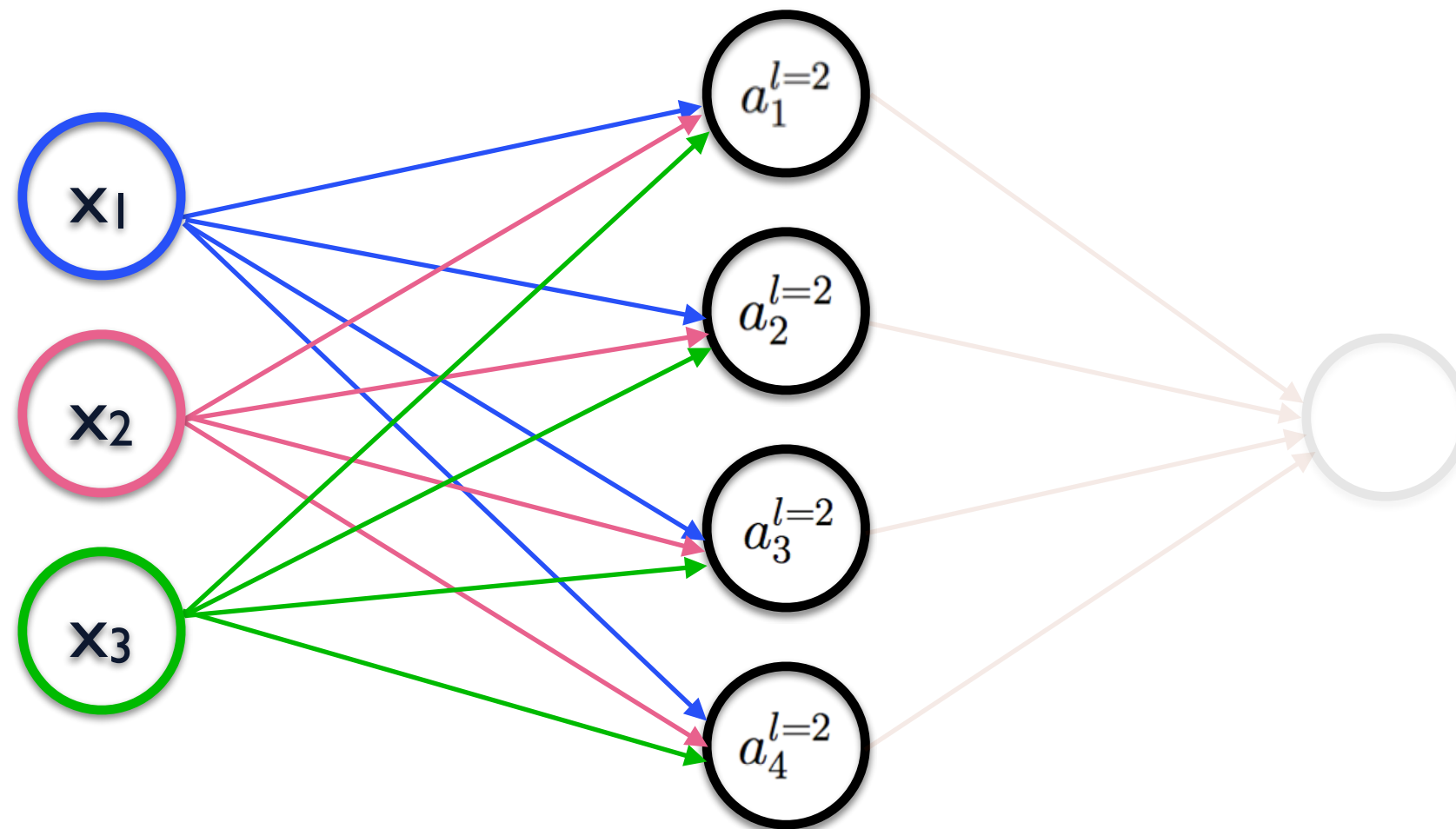
$$a_3^{l=2} = g\left(x_1\theta_{31}^{l=1} + x_2\theta_{32}^{l=1} + x_3\theta_{33}^{l=1}\right)$$
$$a_4^{l=2} = g\left(x_1\theta_{41}^{l=1} + x_2\theta_{42}^{l=1} + x_3\theta_{43}^{l=1}\right)$$

Review: Neural Networks



$$a_3^{l=2} = g\left(x_1\theta_{31}^{l=1} + x_2\theta_{32}^{l=1} + x_3\theta_{33}^{l=1}\right)$$
$$a_4^{l=2} = g\left(x_1\theta_{41}^{l=1} + x_2\theta_{42}^{l=1} + x_3\theta_{43}^{l=1}\right)$$

Review: Neural Networks



$$a_1^{l=2} = g\left(x_1\theta_{11}^{l=1} + x_2\theta_{12}^{l=1} + x_3\theta_{13}^{l=1}\right)$$

$$a_2^{l=2} = g\left(x_1\theta_{21}^{l=1} + x_2\theta_{22}^{l=1} + x_3\theta_{23}^{l=1}\right)$$

$$a_3^{l=2} = g\left(x_1\theta_{31}^{l=1} + x_2\theta_{32}^{l=1} + x_3\theta_{33}^{l=1}\right)$$

$$a_4^{l=2} = g\left(x_1\theta_{41}^{l=1} + x_2\theta_{42}^{l=1} + x_3\theta_{43}^{l=1}\right)$$

Review:

Neural Networks

Cost Function (J)

Review: Neural Networks - Cost Function

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \dots, \theta_{11}^{l=L}, \dots) = \frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1-y^{(i)}) \left(\log(1-f_{\theta}(x^{(i)})) \right)$$

where

$$f_{\theta}(x_1, \dots, x_K) = a^{l=L} = \text{function that computes the network's output (output of the last neuron, in the } L\text{-th layer)}$$

is our hypothesis

We wish to find values for each weight θ that minimize such a cost function, J

Gradient Descent Algorithm

Review: Neural Networks - Cost Function

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \dots, \theta_{11}^{l=L}, \dots) = \frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1-y^{(i)}) \left(\log(1-f_{\theta}(x^{(i)})) \right)$$

where

$f_{\theta}(x_1, \dots, x_K) = a^{l=L} =$ function that computes the network's output (the last neuron, in the L -th layer)

is our hypothesis

Depends only on predicted probabilities ($f_{\theta}(x^{(i)})$)

We wish to find values for each weight θ that minimize such a cost function, J

Independently of how predictions were computed

Gradient Descent Algorithm

Review: Neural Networks - Cost Function

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \dots, \theta_{11}^{l=L}, \dots) = \frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1-y^{(i)}) \left(\log(1-f_{\theta}(x^{(i)})) \right)$$

where

$$f_{\theta}(x_1, \dots, x_K) = a^{l=L} = \text{function that computes the network's output (output of the last neuron, in the } L\text{-th layer)}$$

is our hypothesis

We wish to find values for each weight θ that minimize such a cost function, J

Gradient Descent Algorithm

$$\begin{array}{ll} \frac{\partial J}{\partial \theta_{11}^{l=1}} & \theta_{11}^{l=1} := \theta_{11}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=1}} \\ \frac{\partial J}{\partial \theta_{12}^{l=1}} & \theta_{12}^{l=1} := \theta_{12}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{12}^{l=1}} \\ \vdots & \vdots \\ \frac{\partial J}{\partial \theta_{11}^{l=2}} & \theta_{11}^{l=2} := \theta_{11}^{l=2} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=2}} \\ \vdots & \vdots \\ \frac{\partial J}{\partial \theta_{11}^{l=L}} & \theta_{11}^{l=L} := \theta_{11}^{l=L} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=L}} \end{array}$$

Review: Neural Networks - Cost Function

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \dots, \theta_{11}^{l=L}, \dots) = \frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1-y^{(i)}) \left(\log(1-f_{\theta}(x^{(i)})) \right)$$

where

$$f_{\theta}(x_1, \dots, x_K) = a^{l=L} = \text{function that computes the network's output (output of the last neuron, in the } L\text{-th layer)}$$

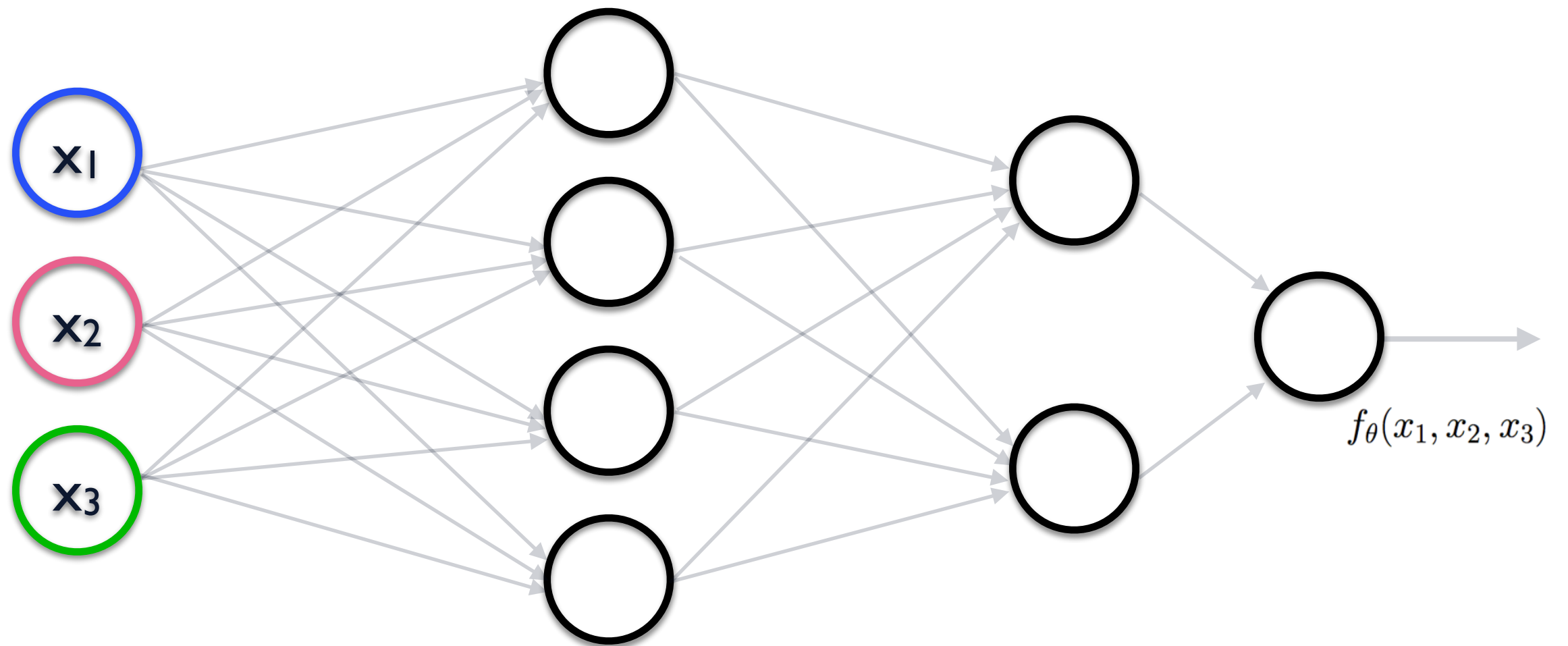
is our hypothesis

We wish to find values for each weight θ that minimize such a cost function, J

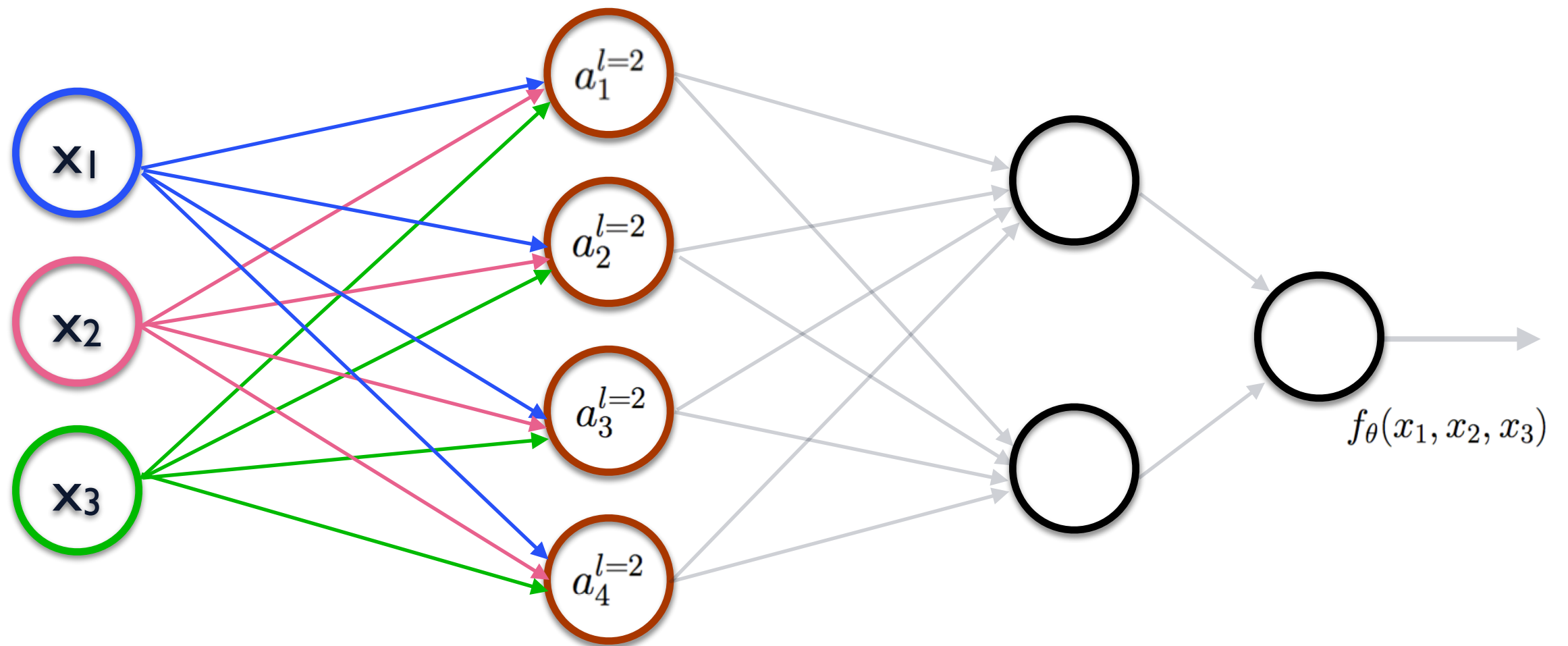
Gradient Descent Algorithm

Backpropagation Algorithm

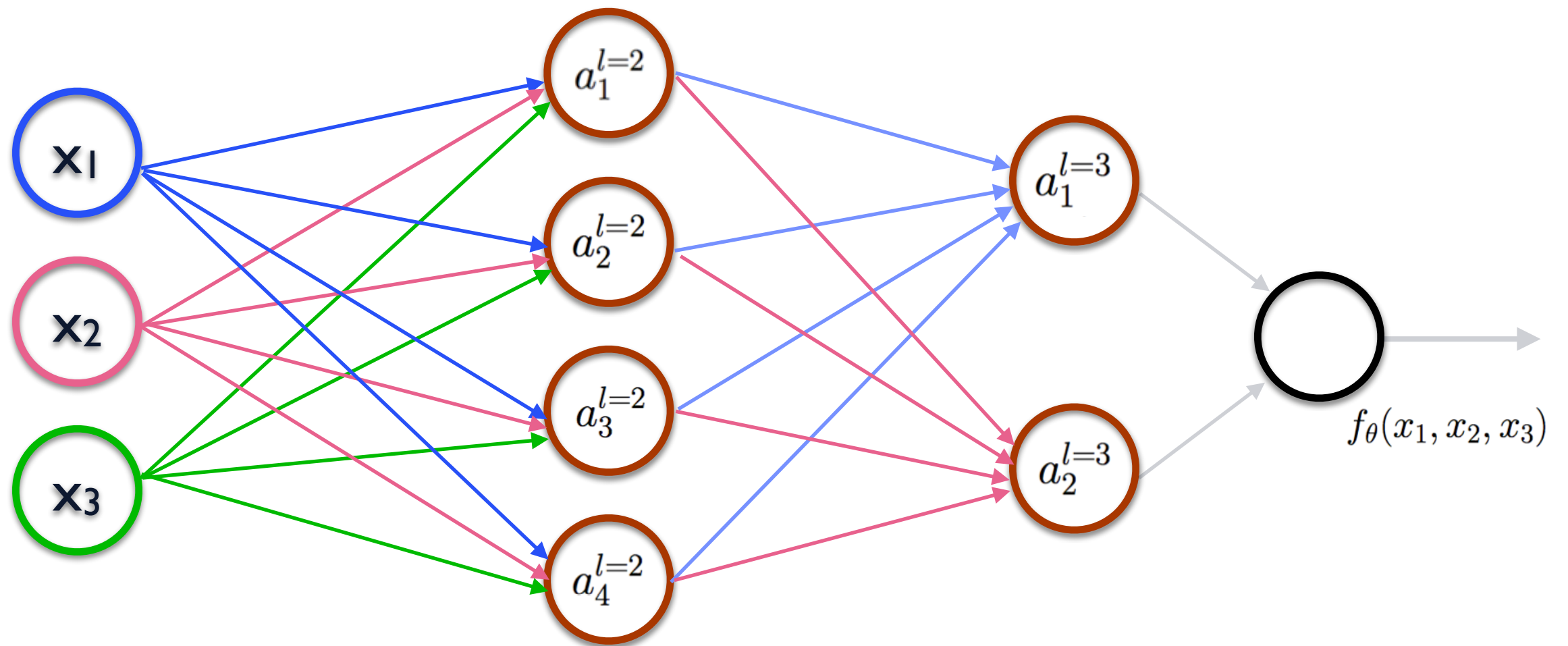
Review: Forward Propagation



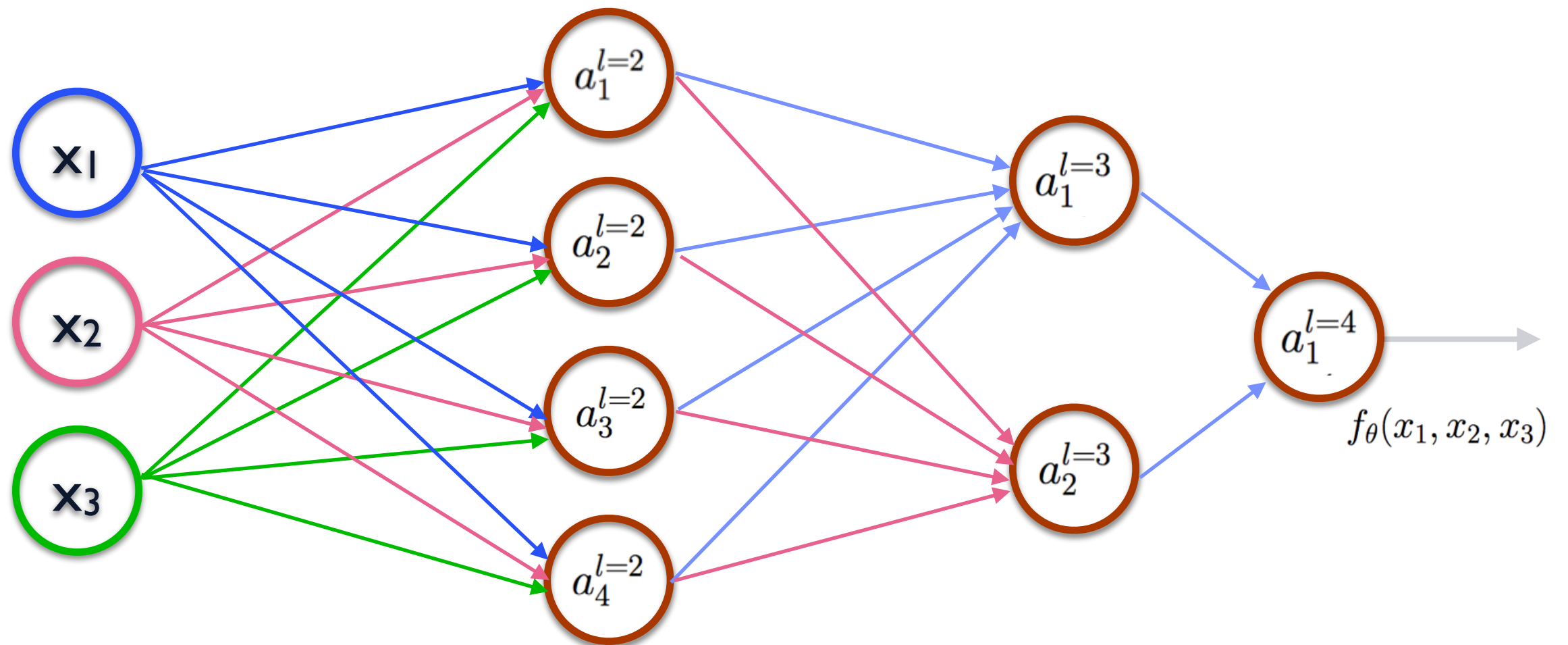
Review: Forward Propagation



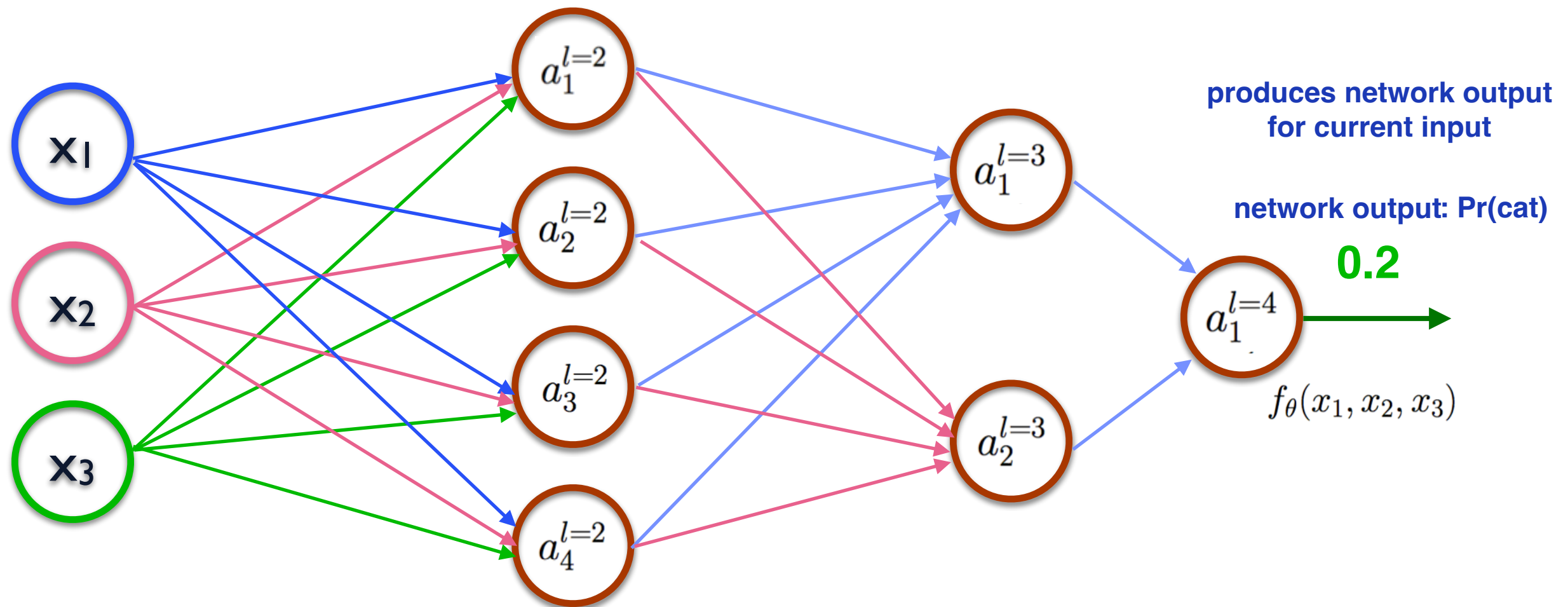
Review: Forward Propagation



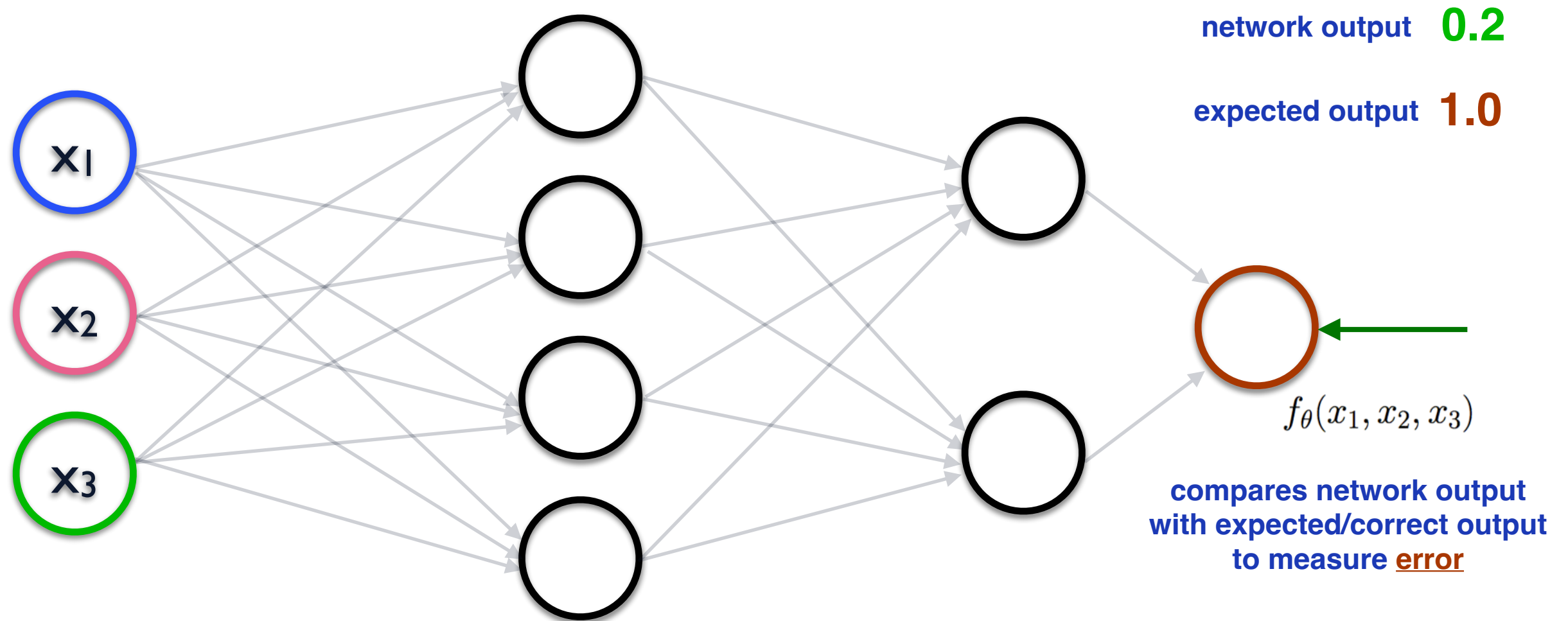
Review: Forward Propagation



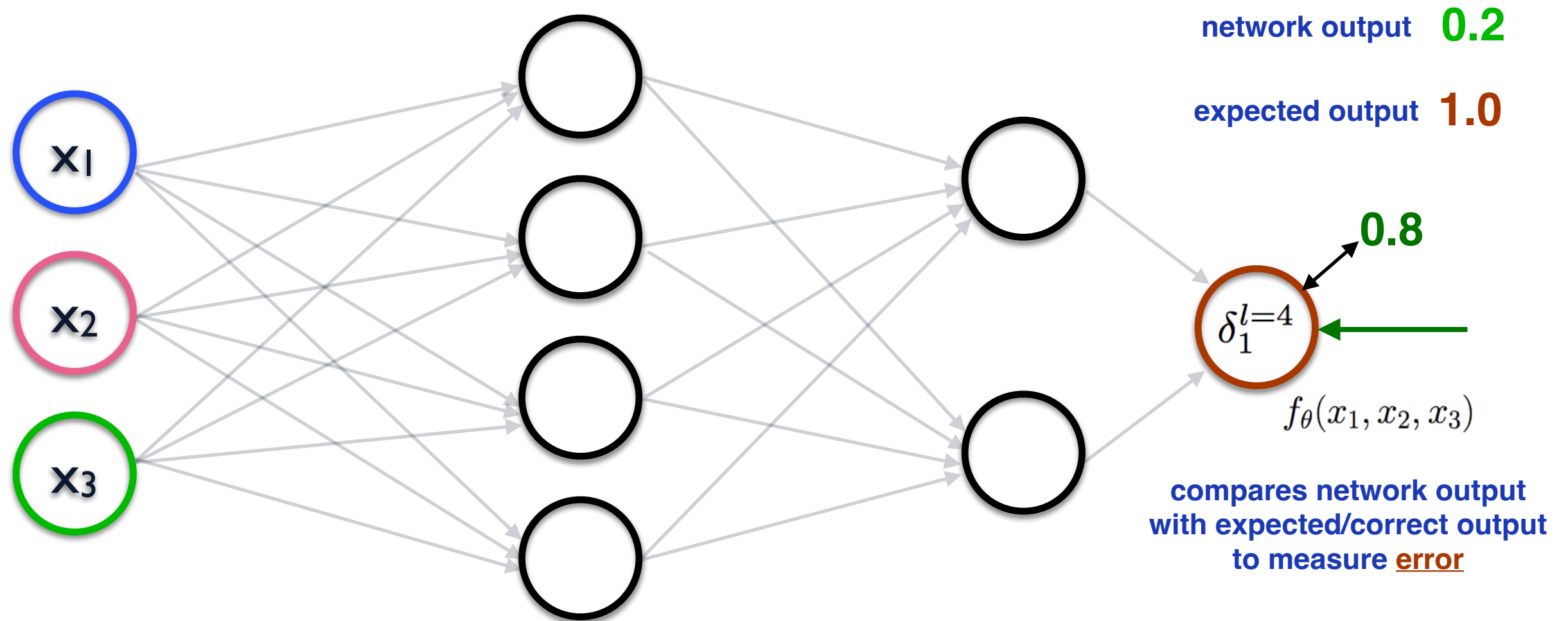
Review: Forward Propagation



Review: Backpropagation of Errors



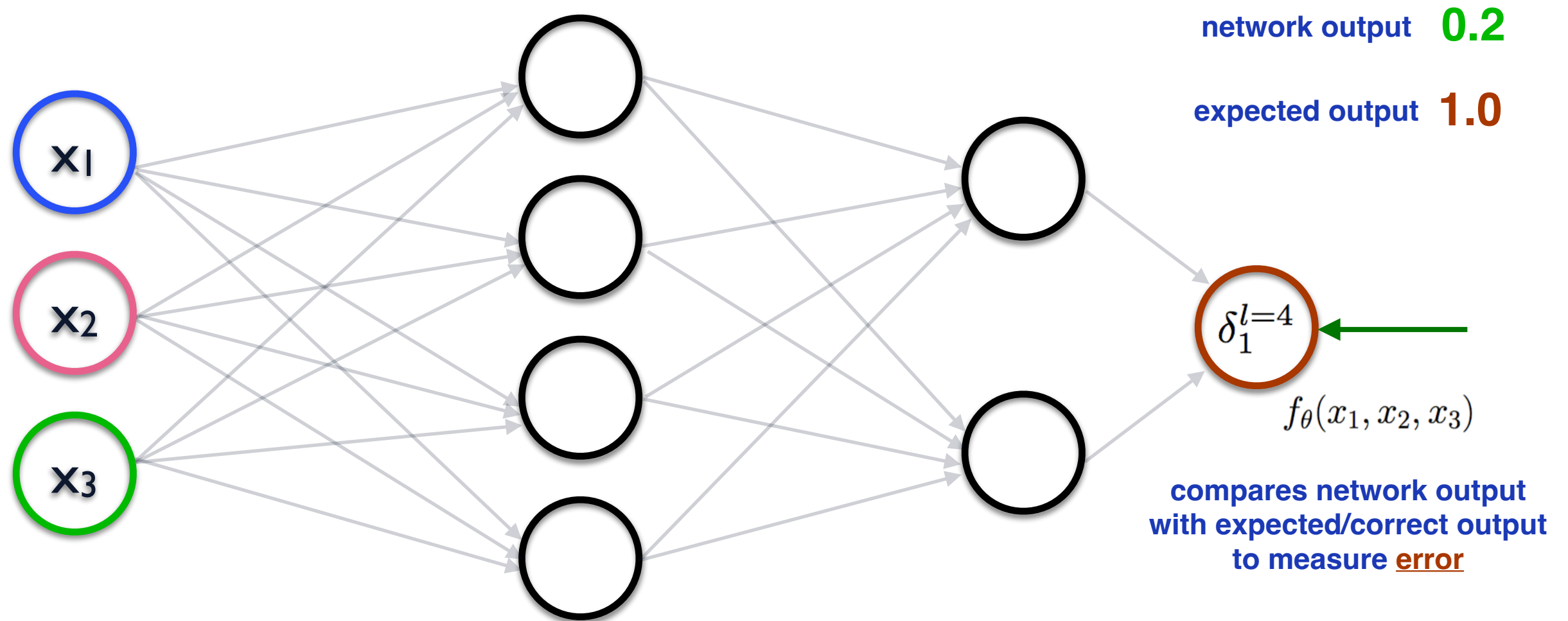
Review: Backpropagation of Errors



Computes a δ value (one per neuron)

Measures how much that neuron's output contributed to the network error

Review: Backpropagation of Errors

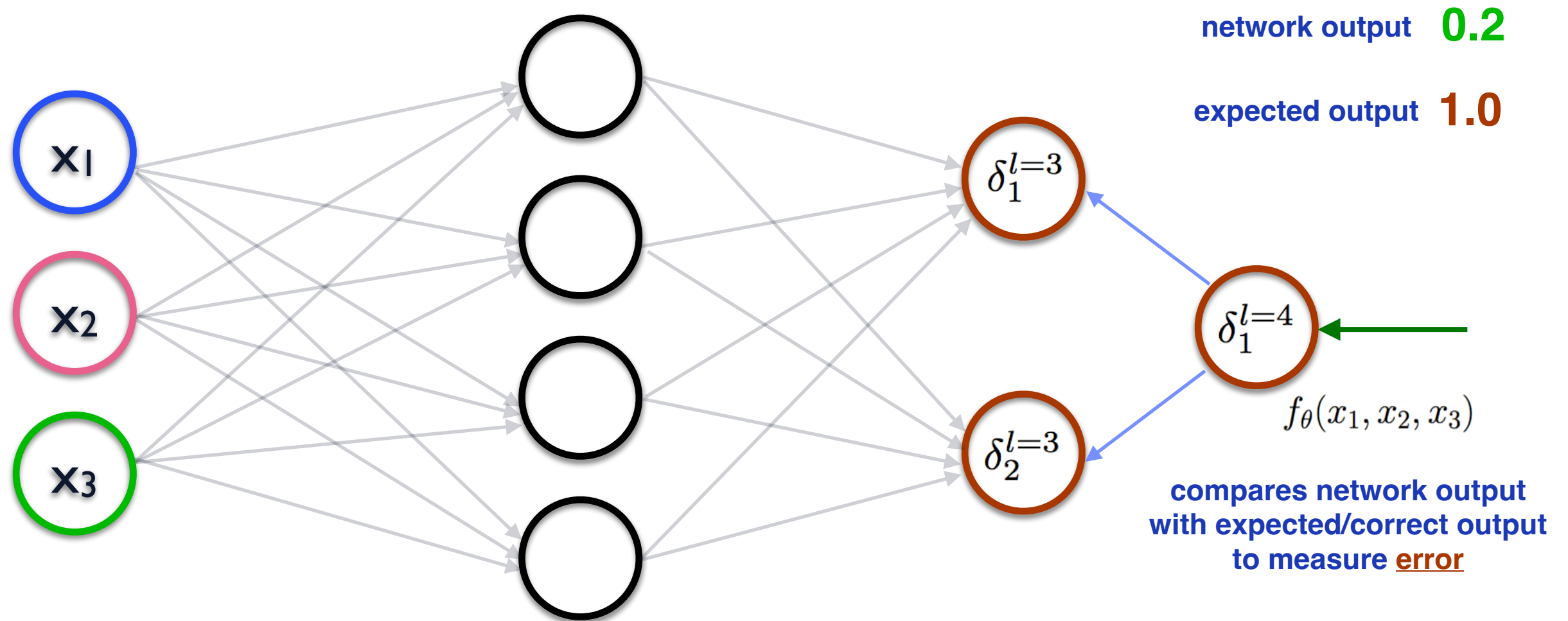


Computes a δ value (one per neuron)

Measures how much that neuron's output contributed to the network error

δ 's are computed from the output layer to the input layer

Review: Backpropagation of Errors

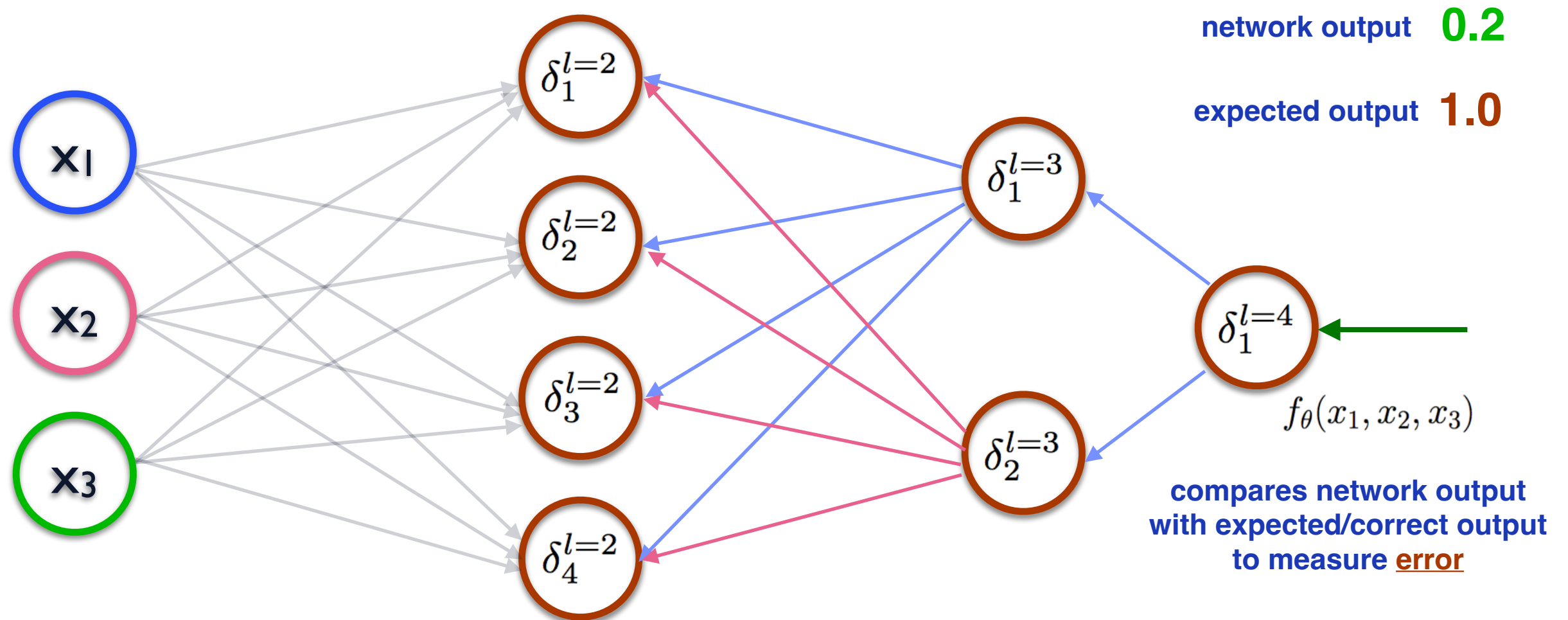


Computes a δ value (one per neuron)

Measures how much that neuron's output contributed to the network error

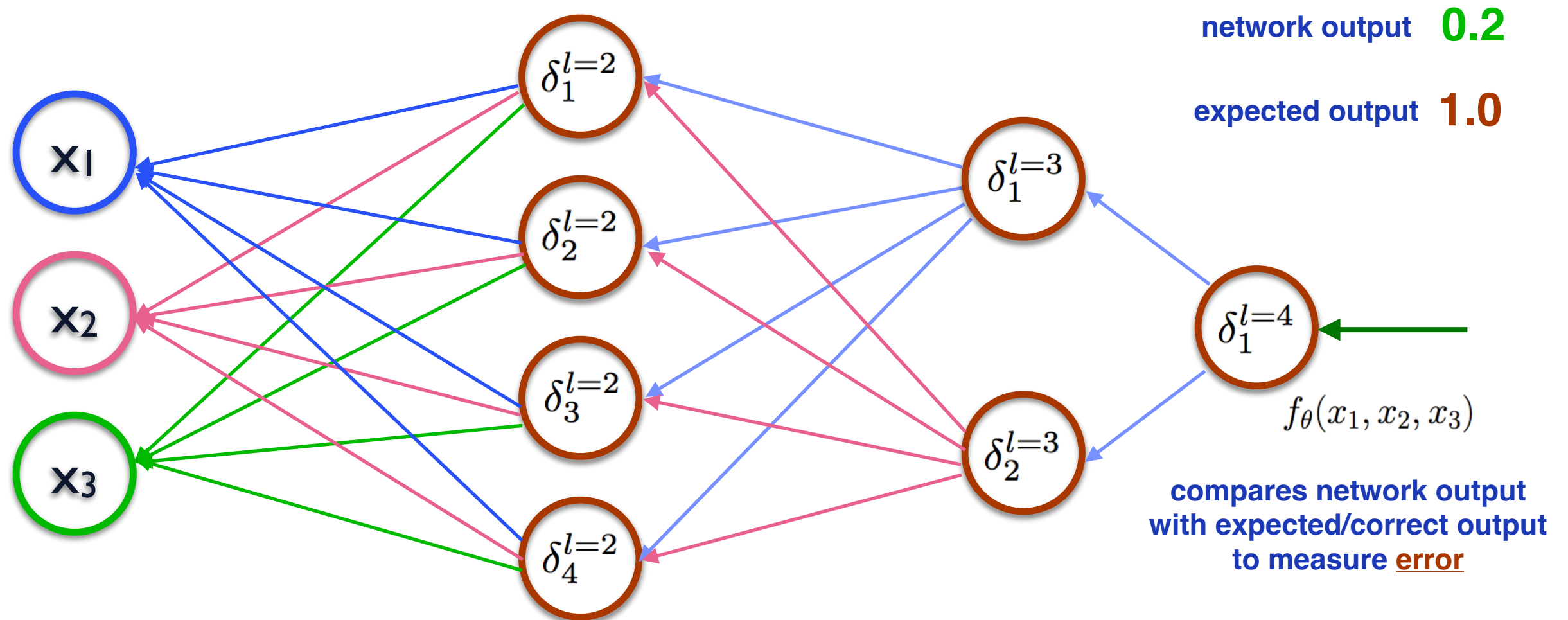
δ 's are computed from the output layer to the input layer

Review: Backpropagation of Errors



δ 's are computed
from the output layer to the input layer

Review: Backpropagation of Errors

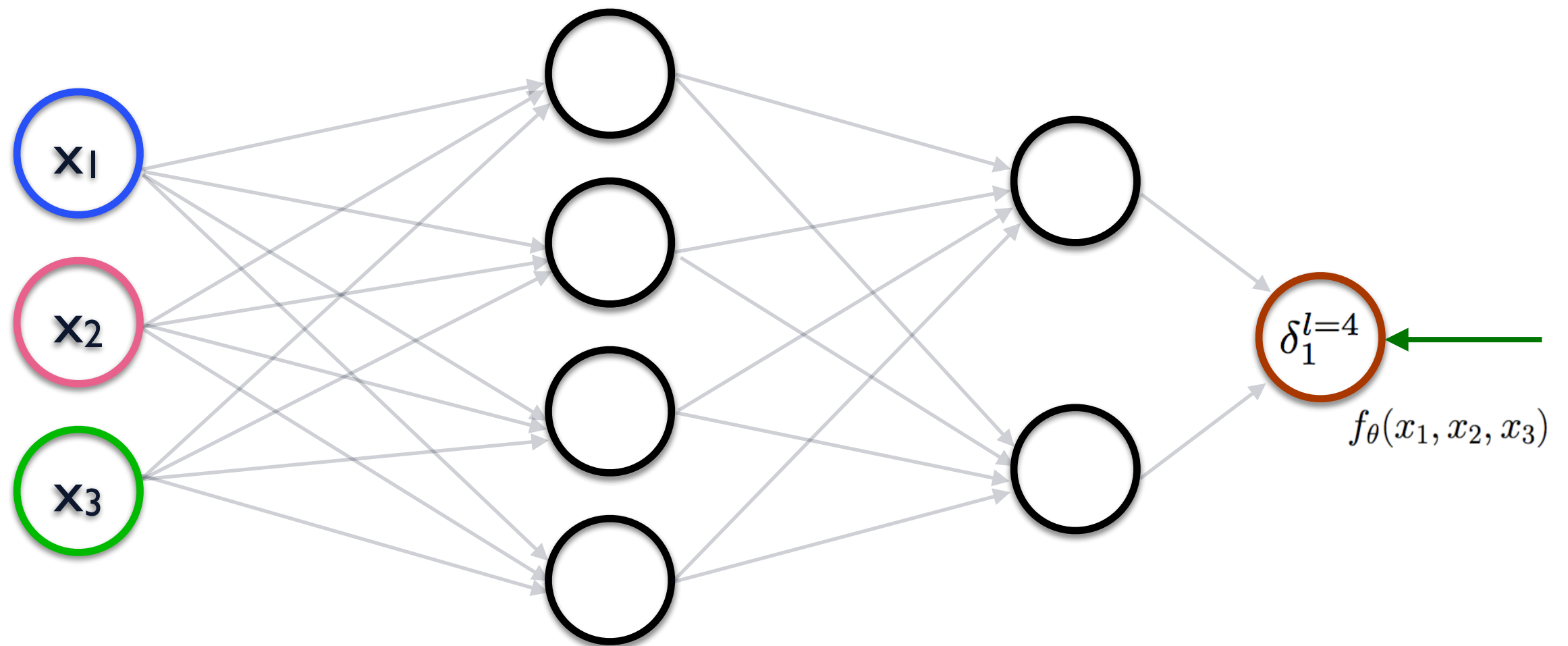


Computes a δ value (one per neuron)

Measures how much that neuron's output contributed to the network error

δ 's are computed
from the output layer to the input layer

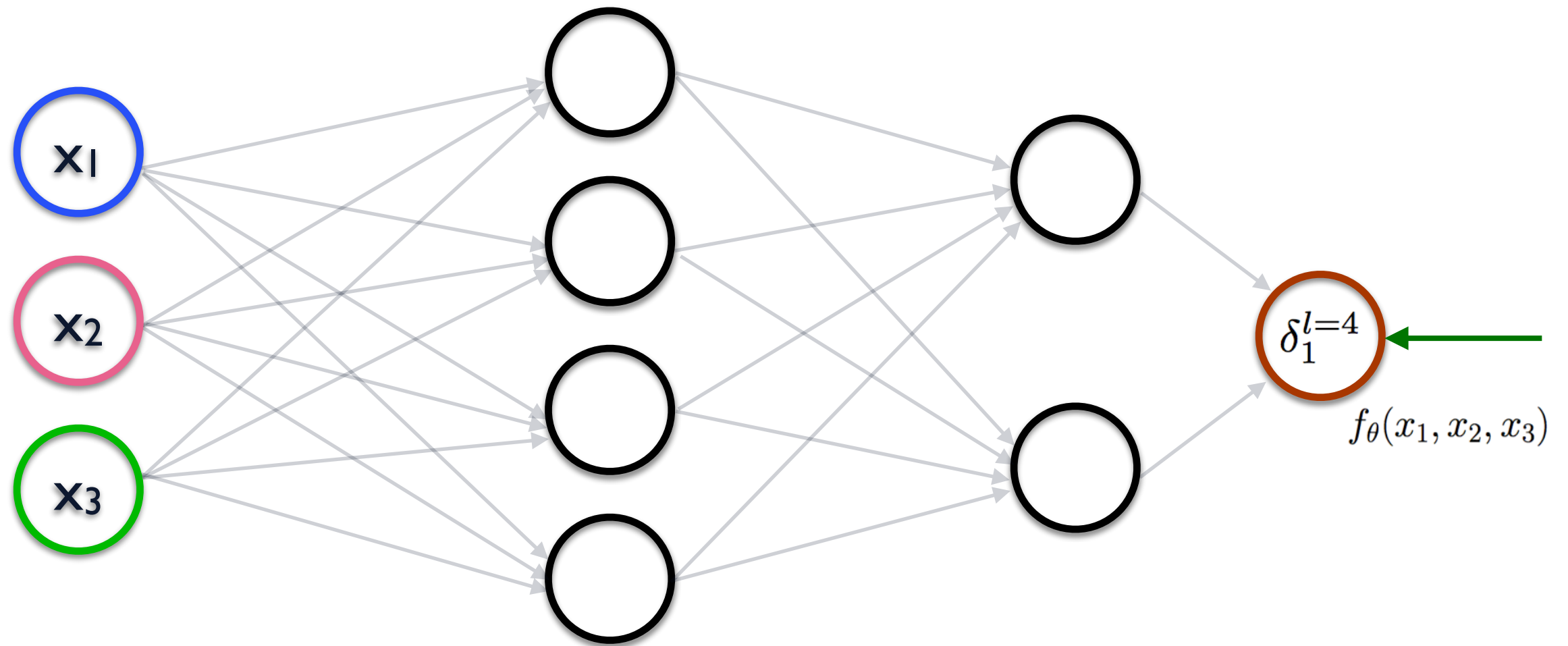
Review: Backpropagation of Errors



δ 's tell us how to update the weights to improve the network

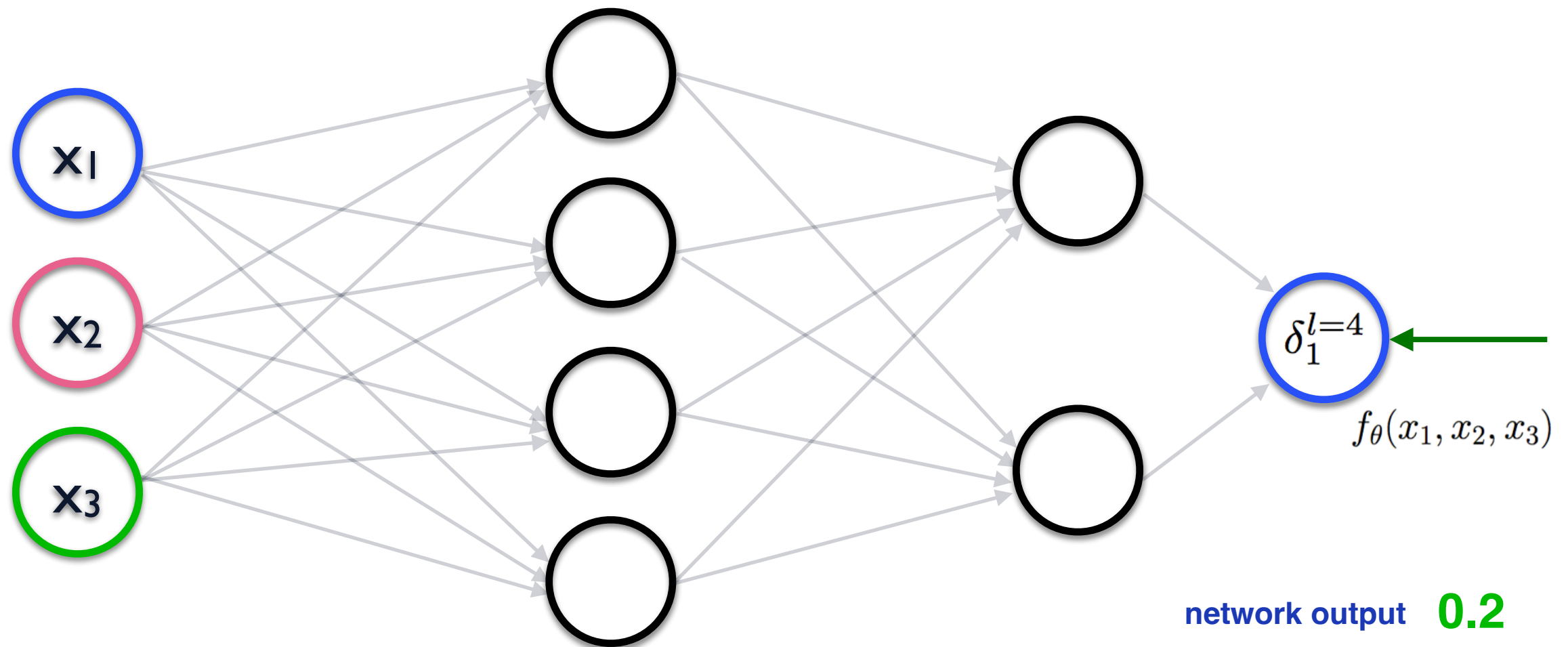
Backpropagation Algorithm

Review: Backpropagation of Errors



How to compute the δ value of each neuron?

Review: Backpropagation of Errors



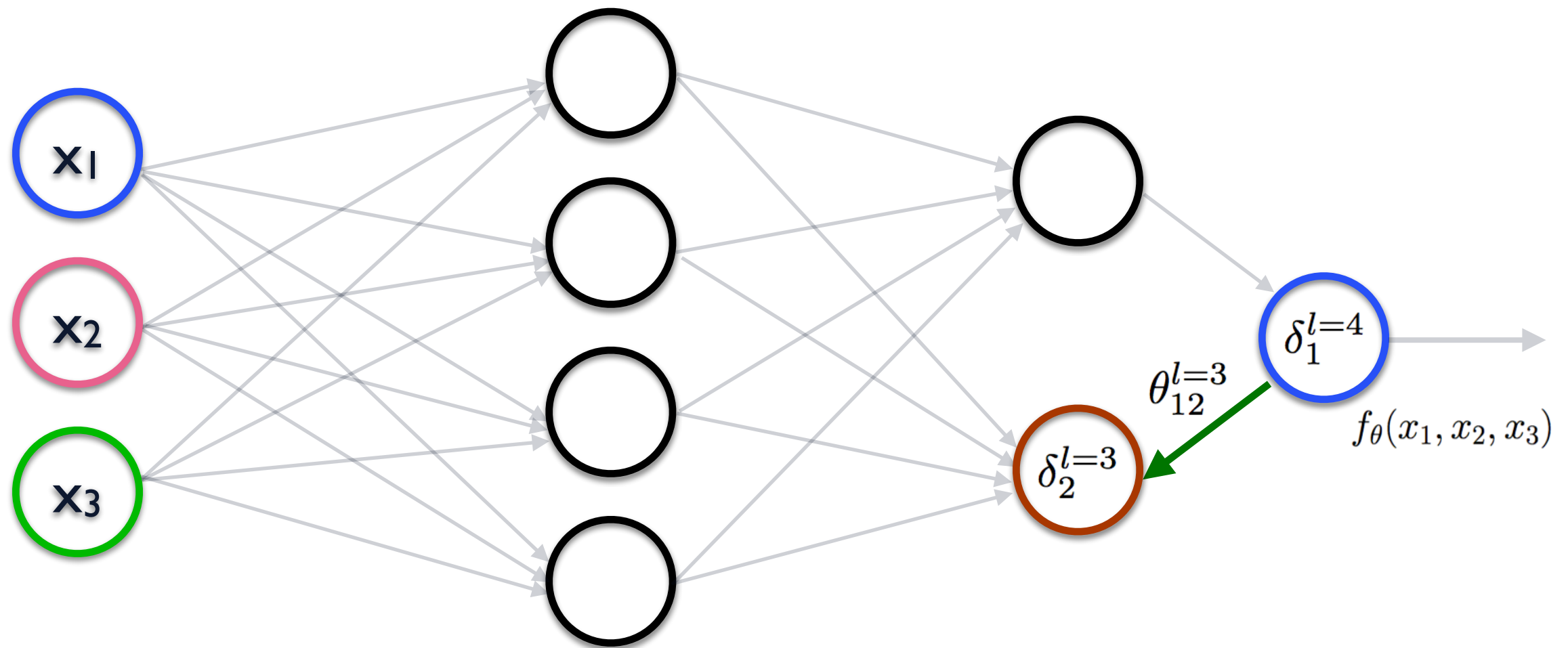
network output **0.2**

expected output **1.0**

$$\delta_1^{l=4} = \left(f_\theta(x_1, x_2, x_3) - y \right) \quad \delta = 0.2 - 1.0 = -0.8$$

δ at the output layer is simply the network's prediction error

Review: Backpropagation of Errors

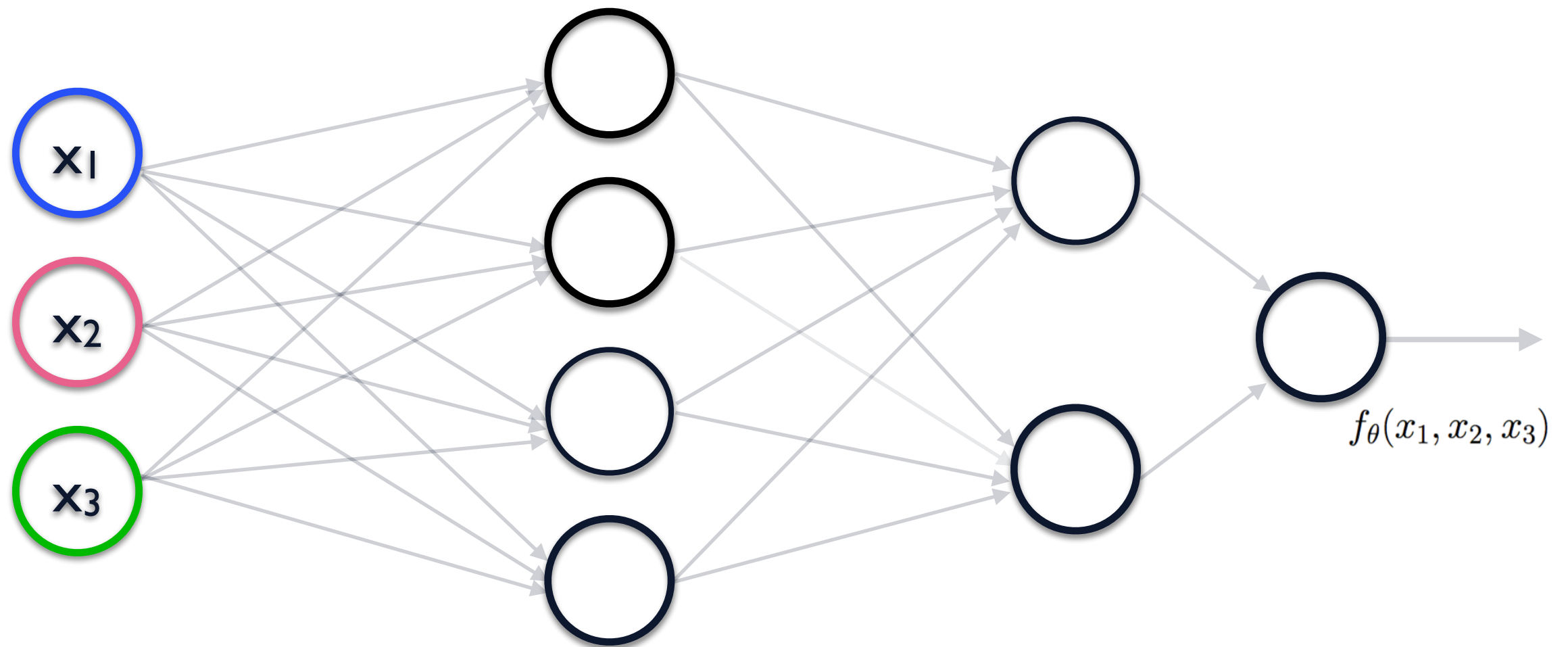


$$\delta_2^{l=3} = \left(\theta_{12}^{l=3} \delta_1^{l=4} \right) \left(a_2^{l=3} \right) \left(1 - a_2^{l=3} \right)$$

δ values of a neuron in a hidden layer:

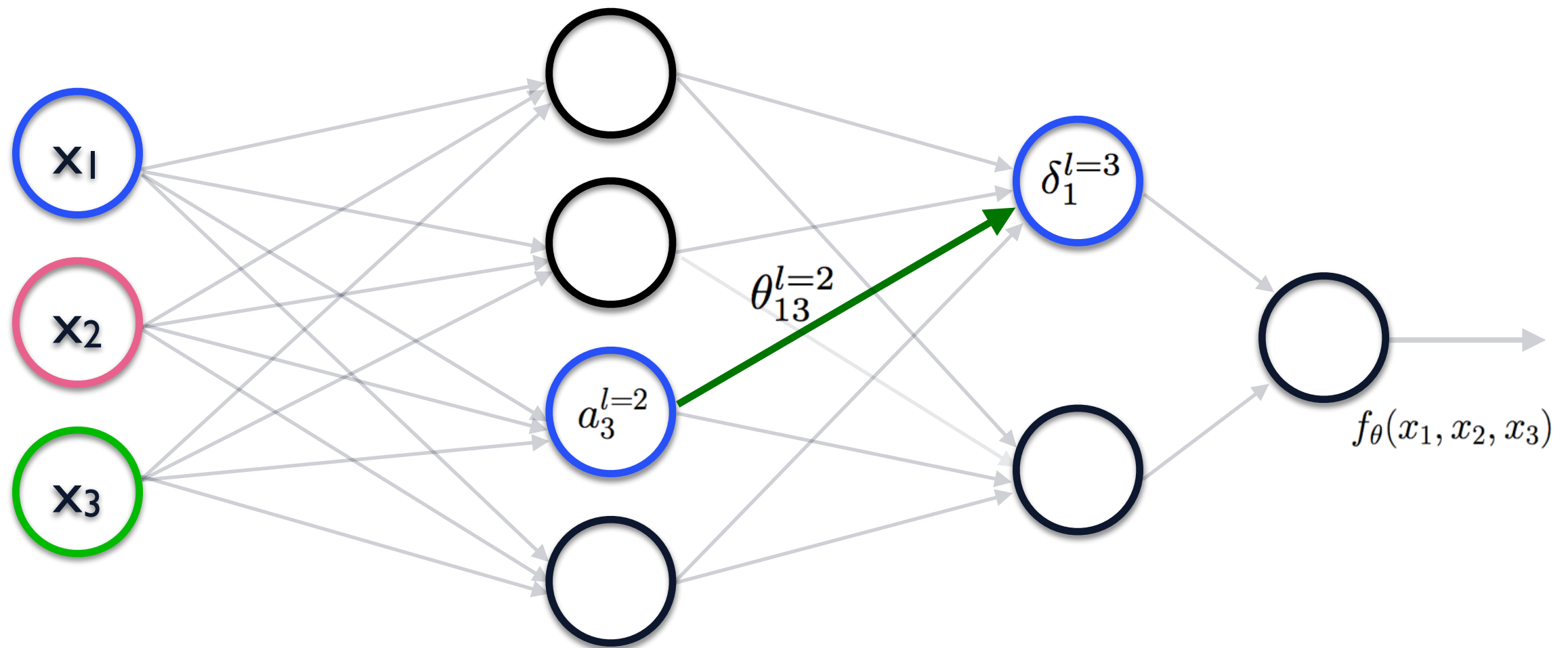
weighted sum of the δ values of neurons in the next layer,
multiplied by the neuron's activation

Review: Backprop - Computing Gradients



**How to compute the gradient of each weight
based on the δ values?**

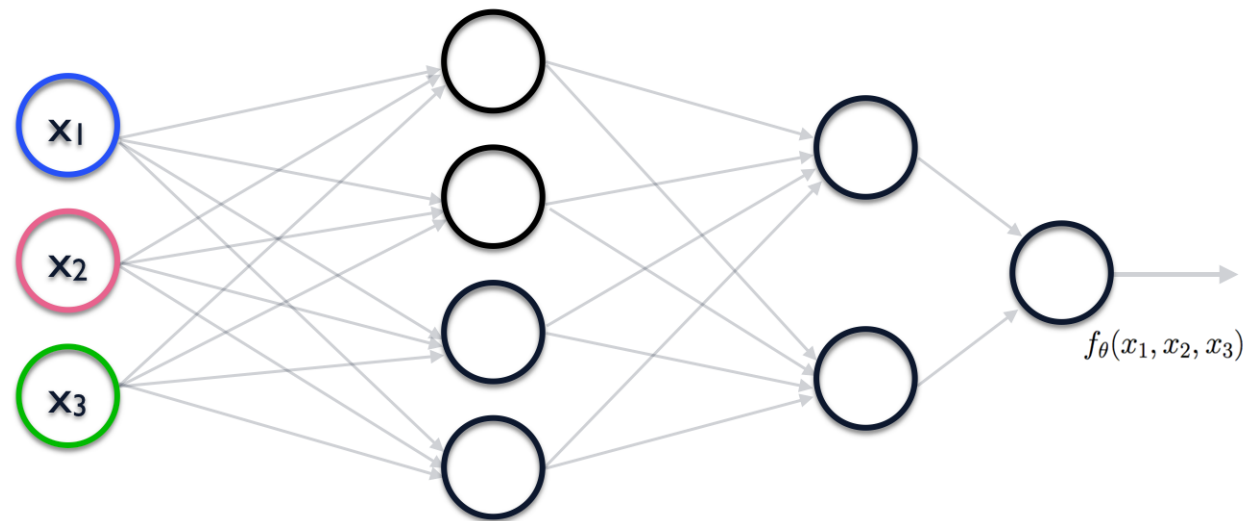
Review: Backprop - Computing Gradients



$$\frac{\partial J}{\partial \theta_{13}^{l=2}} = a_3^{l=2} \delta_1^{l=3}$$

Gradient of a weight depends on
the error made by the network
after processing one instance/input

Backpropagation



δ values for
neurons in the
output layer

$$\delta_1^{l=L} = (f_{\theta}(x) - y)$$

δ values for
neurons in
hidden layers

$$\delta_i^{l=k} = \left(\sum_{j=1}^N \theta_{ij}^{l=k} \delta_j^{l=k+1} \right) (a_i^{l=k}) (1 - a_i^{l=k})$$

(where N is the number of neurons
in the next layer)

gradient of
each weight

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \delta_i^{l=k+1}$$

Backpropagation - Updating Weights

We wish to find values for each weight θ that minimize such a cost function, J

Gradient Descent Algorithm

$$\delta_1^{l=L} = (f_{\theta}(x) - y)$$

$$\delta_i^{l=k} = \left(\sum_{j=1}^N \theta_{ij}^{l=k} \delta_j^{l=k+1} \right) (a_i^{l=k}) (1 - a_i^{l=k})$$

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \delta_i^{l=k+1}$$

$$\frac{\partial J}{\partial \theta_{11}^{l=1}}$$

$$\theta_{11}^{l=1} := \theta_{11}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=1}}$$

$$\frac{\partial J}{\partial \theta_{12}^{l=1}}$$

$$\theta_{12}^{l=1} := \theta_{12}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{12}^{l=1}}$$

$$\vdots$$

$$\vdots$$

$$\frac{\partial J}{\partial \theta_{11}^{l=2}}$$

$$\theta_{11}^{l=2} := \theta_{11}^{l=2} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=2}}$$

$$\vdots$$

$$\vdots$$

$$\frac{\partial J}{\partial \theta_{11}^{l=L}}$$

$$\theta_{11}^{l=L} := \theta_{11}^{l=L} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=L}}$$

Backpropagation Algorithm

1. Initialize all weights of the network with non-zero random values
2. For each instance $(\mathbf{x}, \mathbf{y})$ in the training set:
 1. Propagate the instance through the network, computing its output/prediction $\mathbf{f}_\theta(\mathbf{x})$
 2. Compute the δ values for all neurons in the output layer

$$\delta_1^{l=L} = (f_\theta(x) - y)$$

3. Compute the δ values for all neurons in the hidden layers:

$$\delta_i^{l=k} = \left(\sum_{j=1}^N \theta_{ij}^{l=k} \delta_j^{l=k+1} \right) (a_i^{l=k}) (1 - a_i^{l=k})$$

4. Computes the gradients of all weights of the network:

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \delta_i^{l=k+1}$$

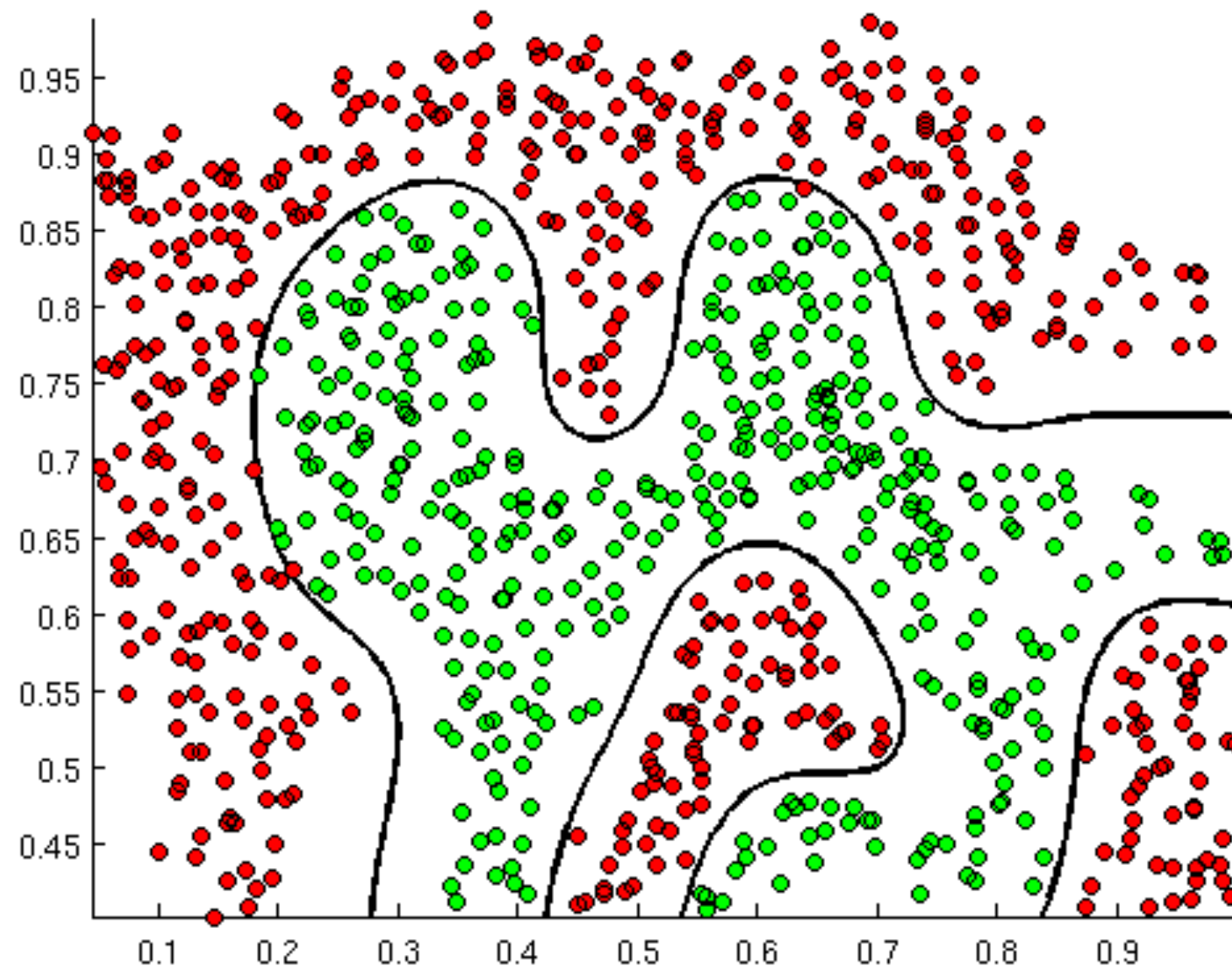
5. Adjust all weights of the network according to the gradient descent update rule:

$$\theta_{ij}^l := \theta_{ij}^l - \alpha \frac{\partial J}{\partial \theta_{ij}^l}$$

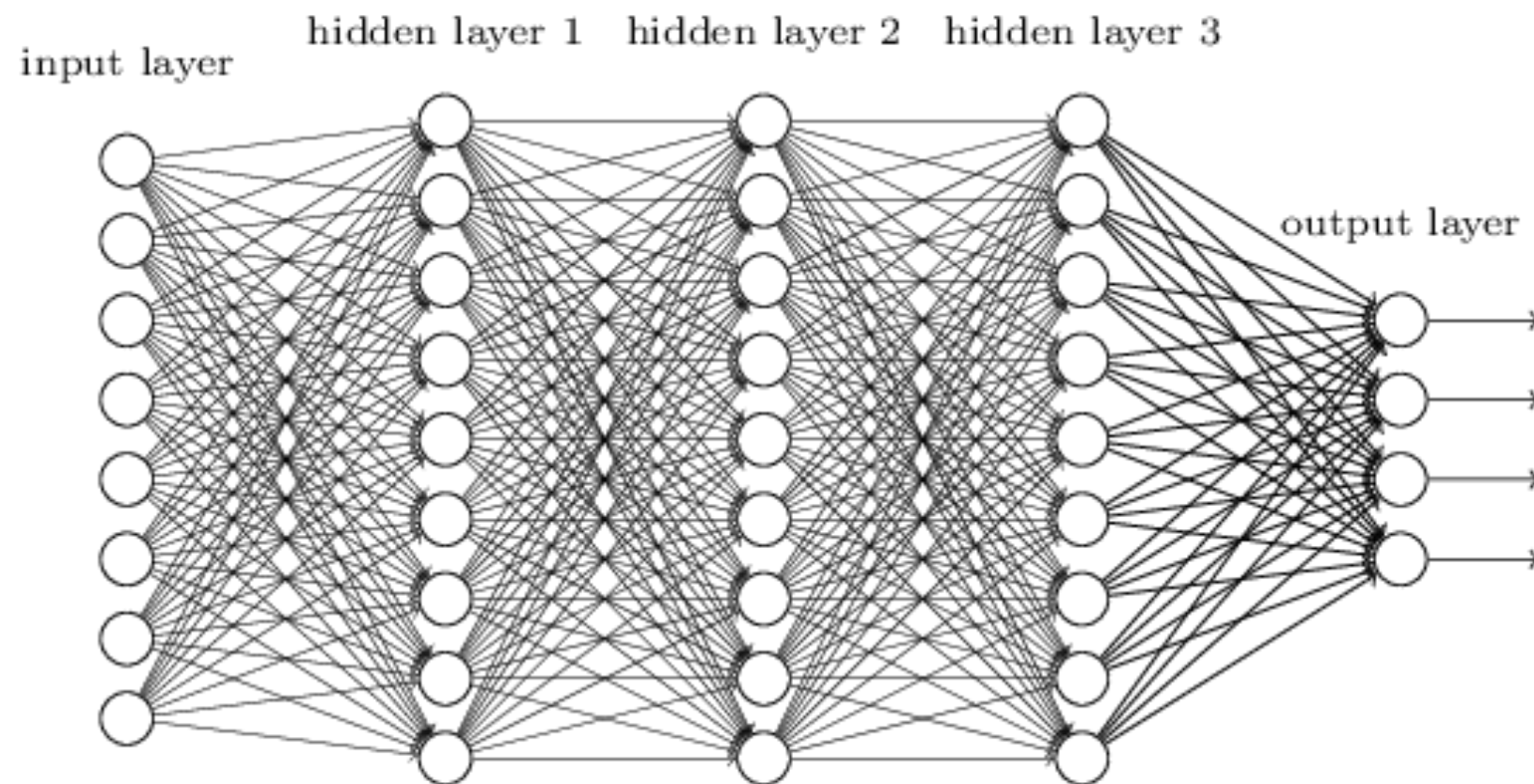
3. Evaluate the performance, $\mathbf{J}$, of the network

1. If sufficiently good (or if improvement is too small), then stop

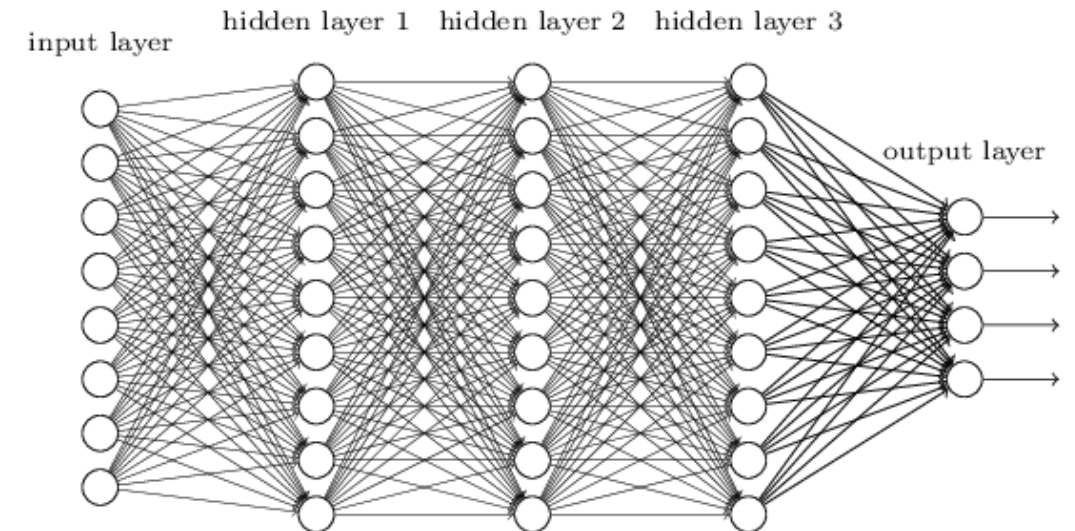
Backpropagation - Decision Boundaries



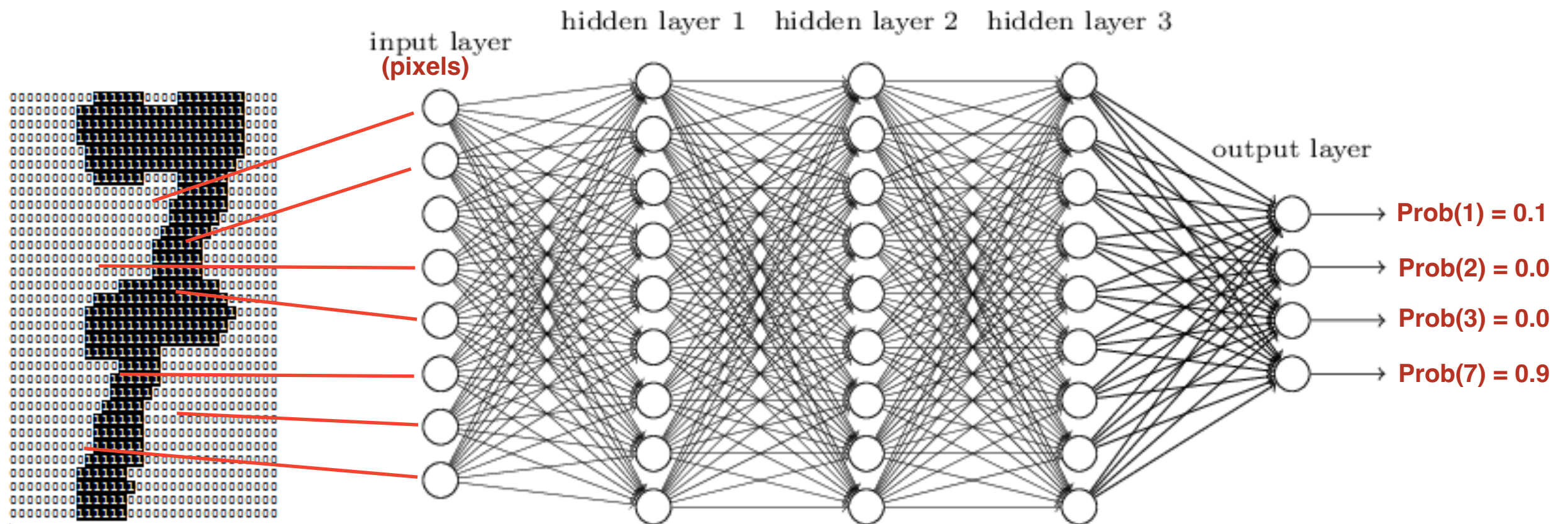
Handwriting Recognition



Handwriting Recognition



Handwriting Recognition



Real-Time Identification/Classification of Objects



https://www.youtube.com/watch?v=n5uP_LP9SmM

Backpropagation

Demo

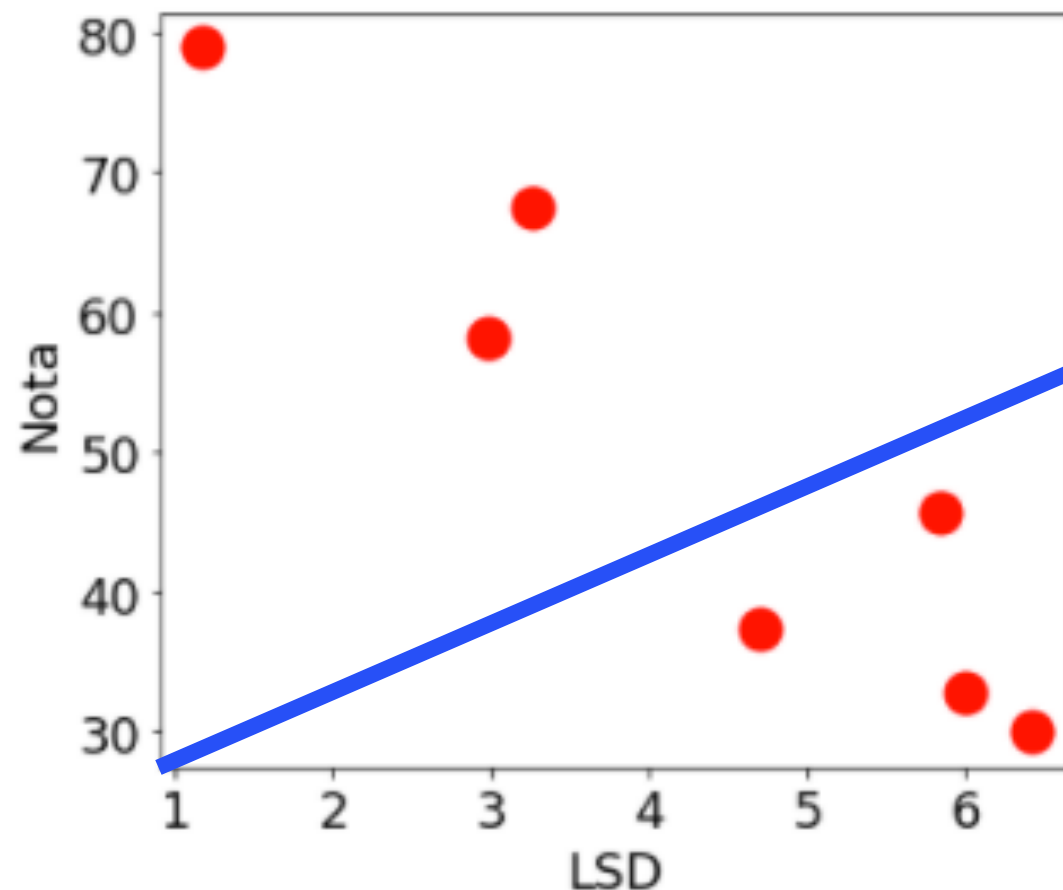
playground.tensorflow.org

Bias Term

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$

where $f_{\theta}(x) = \theta_0 + \theta_1 x$ is our hypothesis



Why do we need this θ_0 term?

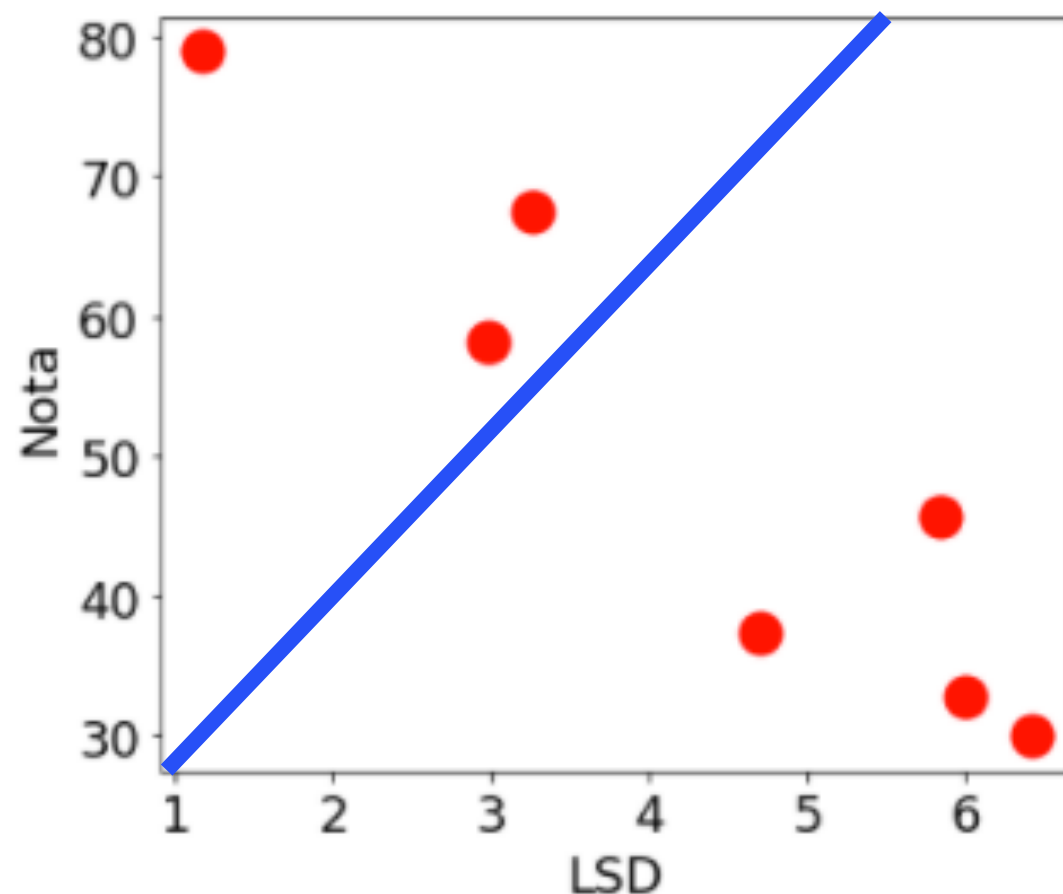
$$f_{\theta}(x) = 14x$$

Bias Term

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$

where $f_{\theta}(x) = \theta_0 + \theta_1 x$ is our hypothesis



Why do we need this θ_0 term?

Only hypotheses that pass through the origin (0,0)

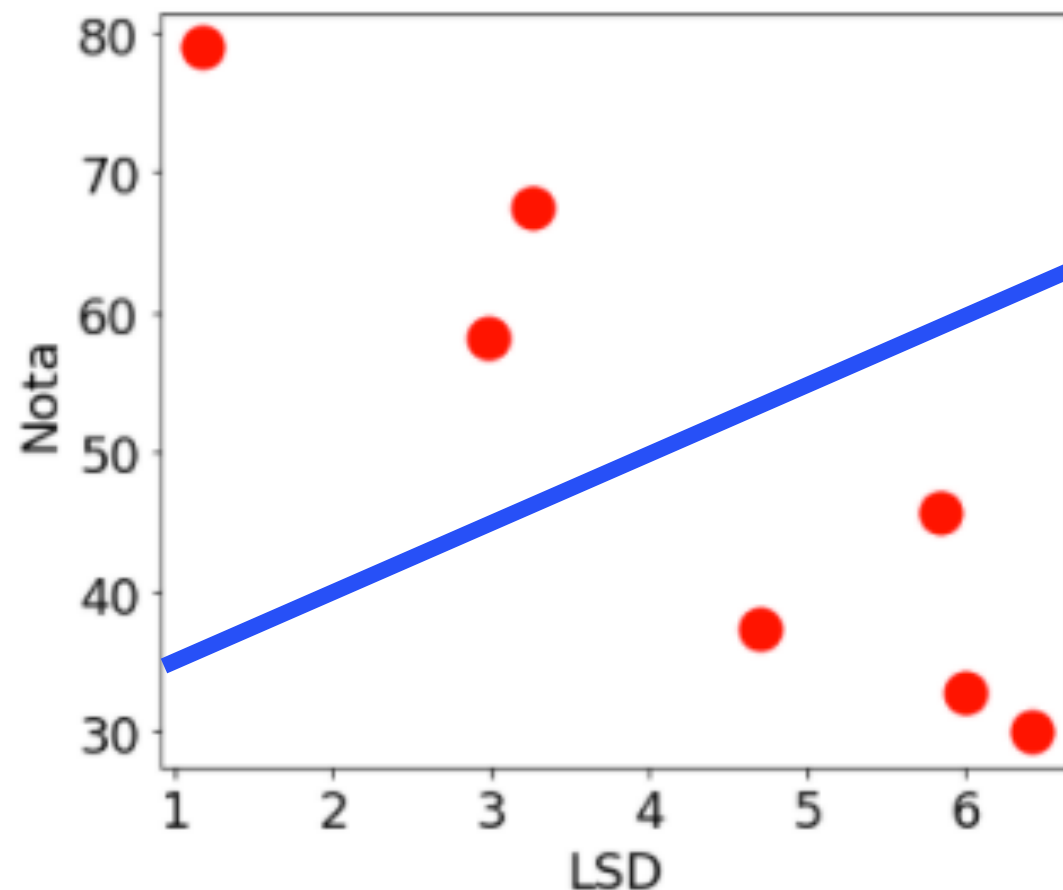
$$f_{\theta}(x) = 20x$$

Bias Term

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$

where $f_{\theta}(x) = \theta_0 + \theta_1 x$ is our hypothesis



Why do we need this θ_0 term?

Only hypotheses that pass through the origin (0,0)

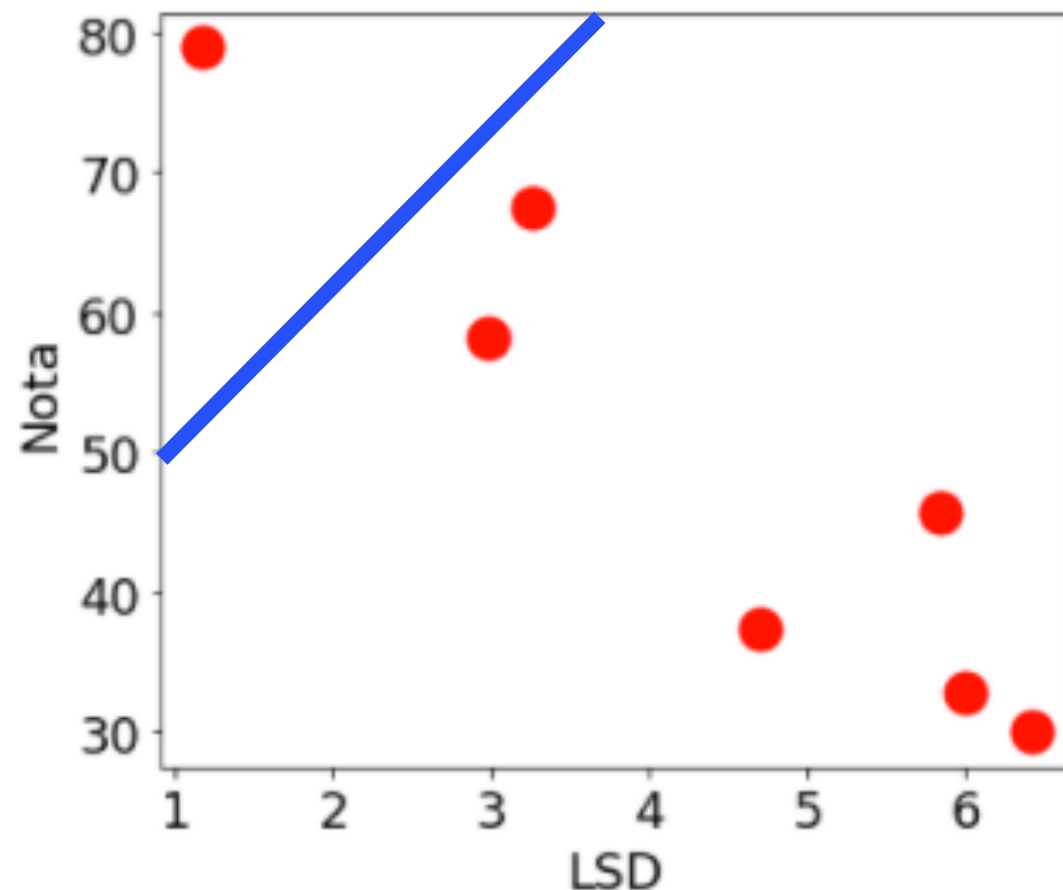
$$f_{\theta}(x) = 35 + 14x$$

Bias Term

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$

where $f_{\theta}(x) = \theta_0 + \theta_1 x$ is our hypothesis



Why do we need this θ_0 term?

Only hypotheses that pass through the origin (0,0)

$$f_{\theta}(x) = 50 + 20x$$

Bias Term

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2 \quad \text{where } f_{\theta}(x) = \theta_0 + \theta_1 x \quad \text{is our hypothesis}$$

Logistic Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1, \theta_2) = \frac{1}{n} \sum_{i=1}^n y^{(i)} \left(-\log(f_{\theta}(x_1^{(i)}, x_2^{(i)})) \right) - (1 - y^{(i)}) \left(\log(1 - f_{\theta}(x_1^{(i)}, x_2^{(i)})) \right)$$

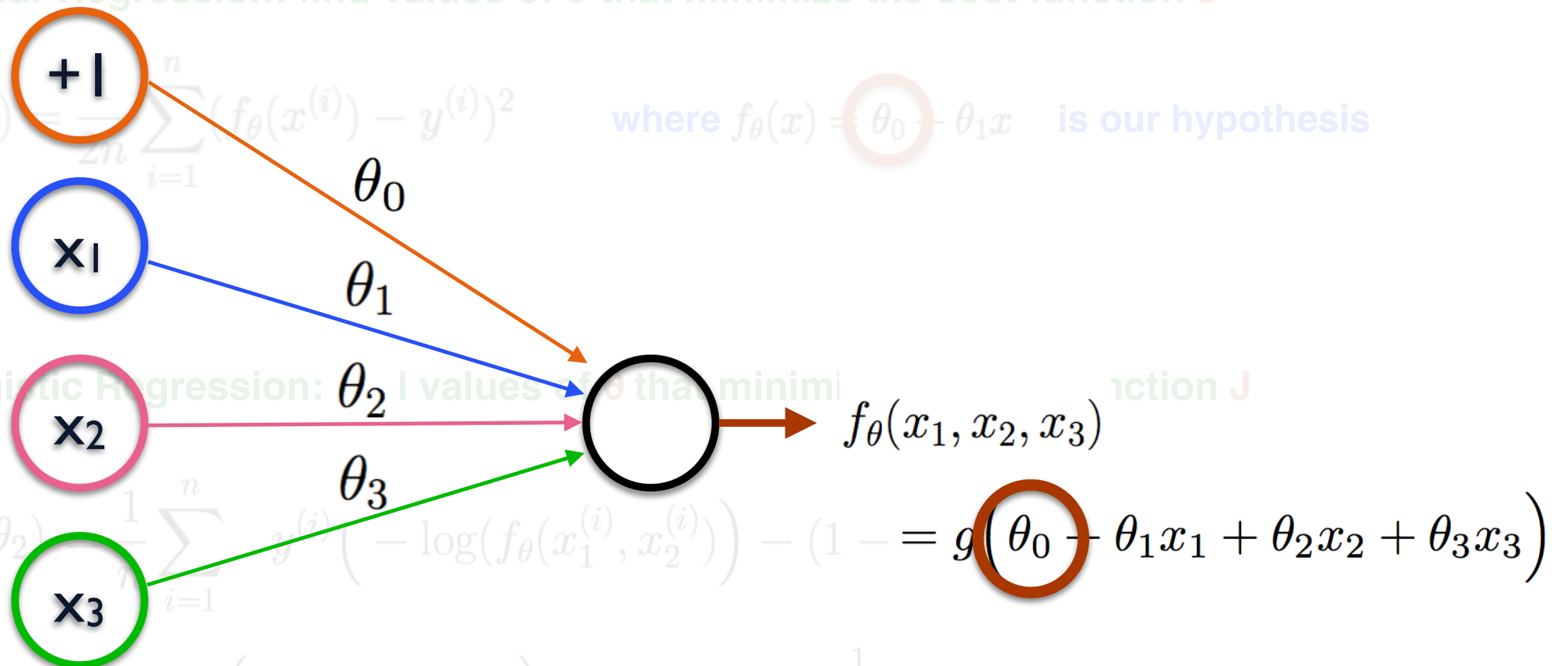
where $f_{\theta}(x_1, x_2) = g\left(\theta_0 + \theta_1 x_1 + \theta_2 x_2\right) = \frac{1}{1 + \exp\left(-(\theta_0 + \theta_1 x_1 + \theta_2 x_2)\right)}$ **is our hypothesis**

**These terms are known as *bias* terms
Where do they show up in neural networks?**

Bias Term

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2 \quad \text{where } f_{\theta}(x) = \theta_0 + \theta_1 x \text{ is our hypothesis}$$



$$J(\theta_0, \theta_1, \theta_2) = \frac{1}{2n} \sum_{i=1}^n (y^{(i)} - \log(f_{\theta}(x_1^{(i)}, x_2^{(i)})) - (1 - \log(1 - f_{\theta}(x_1^{(i)}, x_2^{(i)}))) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3)$$

$$\text{where } f_{\theta}(x_1, x_2) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2) = \frac{1}{1 + \exp(-(\theta_0 + \theta_1 x_1 + \theta_2 x_2))} \text{ is our hypothesis}$$

These terms are known as *bias* terms
Where do they show up in neural networks?

Bias Term

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2 \quad \text{where } f_{\theta}(x) = \theta_0 + \theta_1 x \quad \text{is our hypothesis}$$

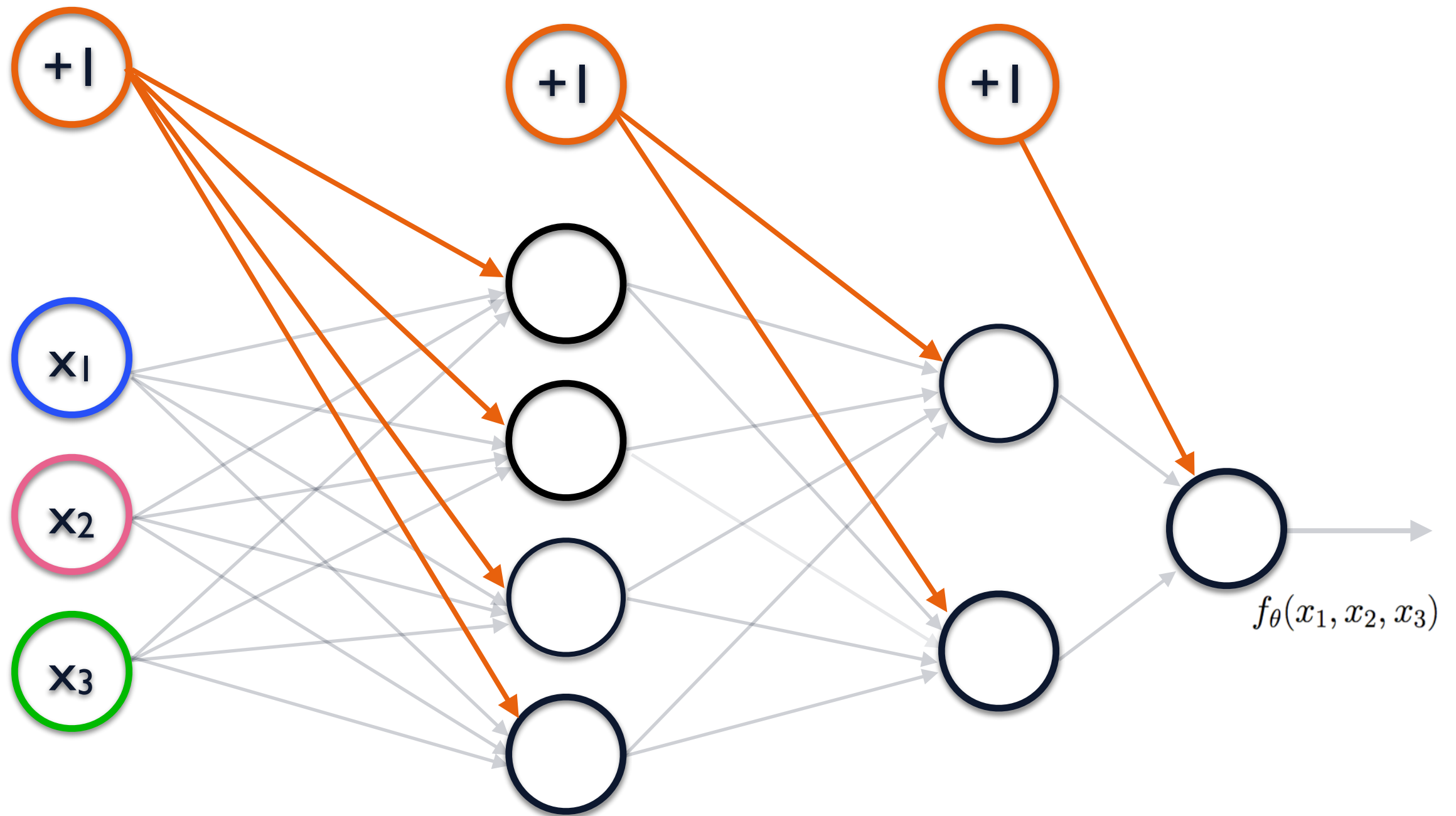
Logistic Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1, \theta_2) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} \left(-\log(f_{\theta}(x_1^{(i)}, x_2^{(i)})) \right) - (1 - y^{(i)}) \left(-\log(1 - f_{\theta}(x_1^{(i)}, x_2^{(i)})) \right) \right)$$

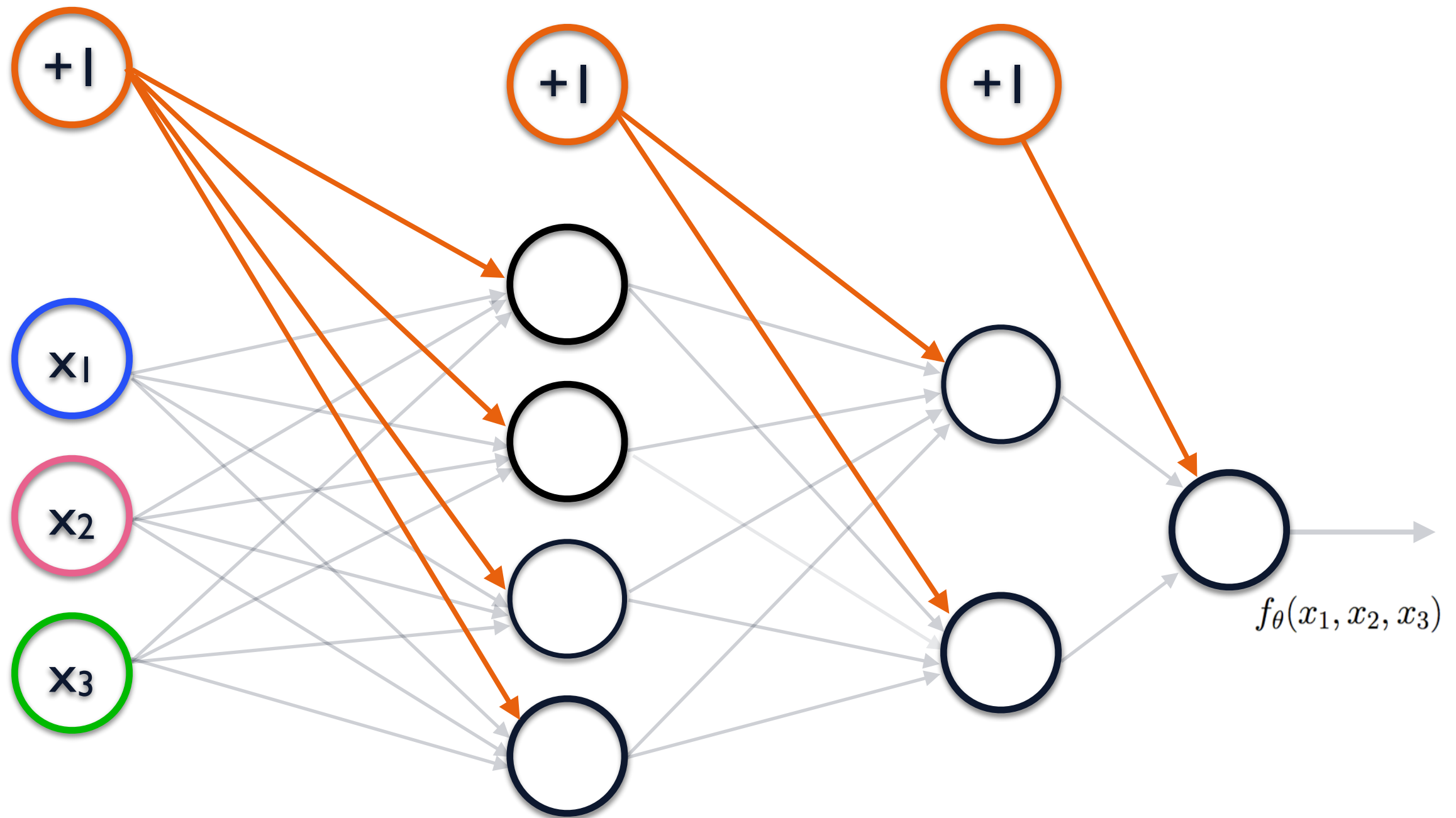
$$f_{\theta}(x_1, x_2, x_3) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3)$$

$$\text{where } f_{\theta}(x_1, x_2) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2) = \frac{1}{1 + \exp(-(\theta_0 + \theta_1 x_1 + \theta_2 x_2))} \quad \text{is our hypothesis}$$

These terms are known as **bias terms**
 Added as “extra” inputs to each neuron



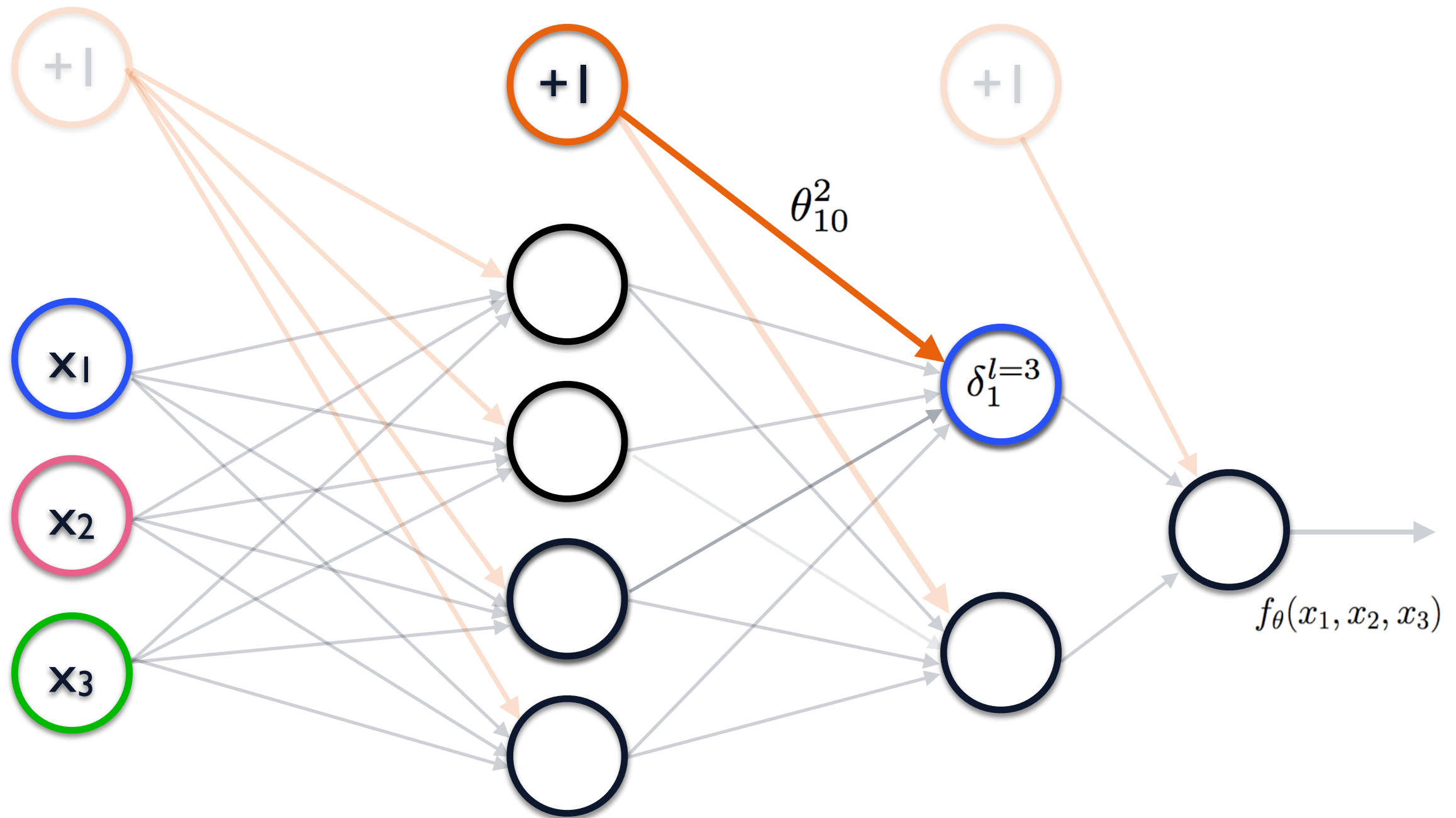
**How to update the weights
of the “*bias* neurons”?**



Values of δ value (one per neuron)
are computed and then used
to update the
weights **arriving** at that neuron

But “bias neurons” have
no input connections!

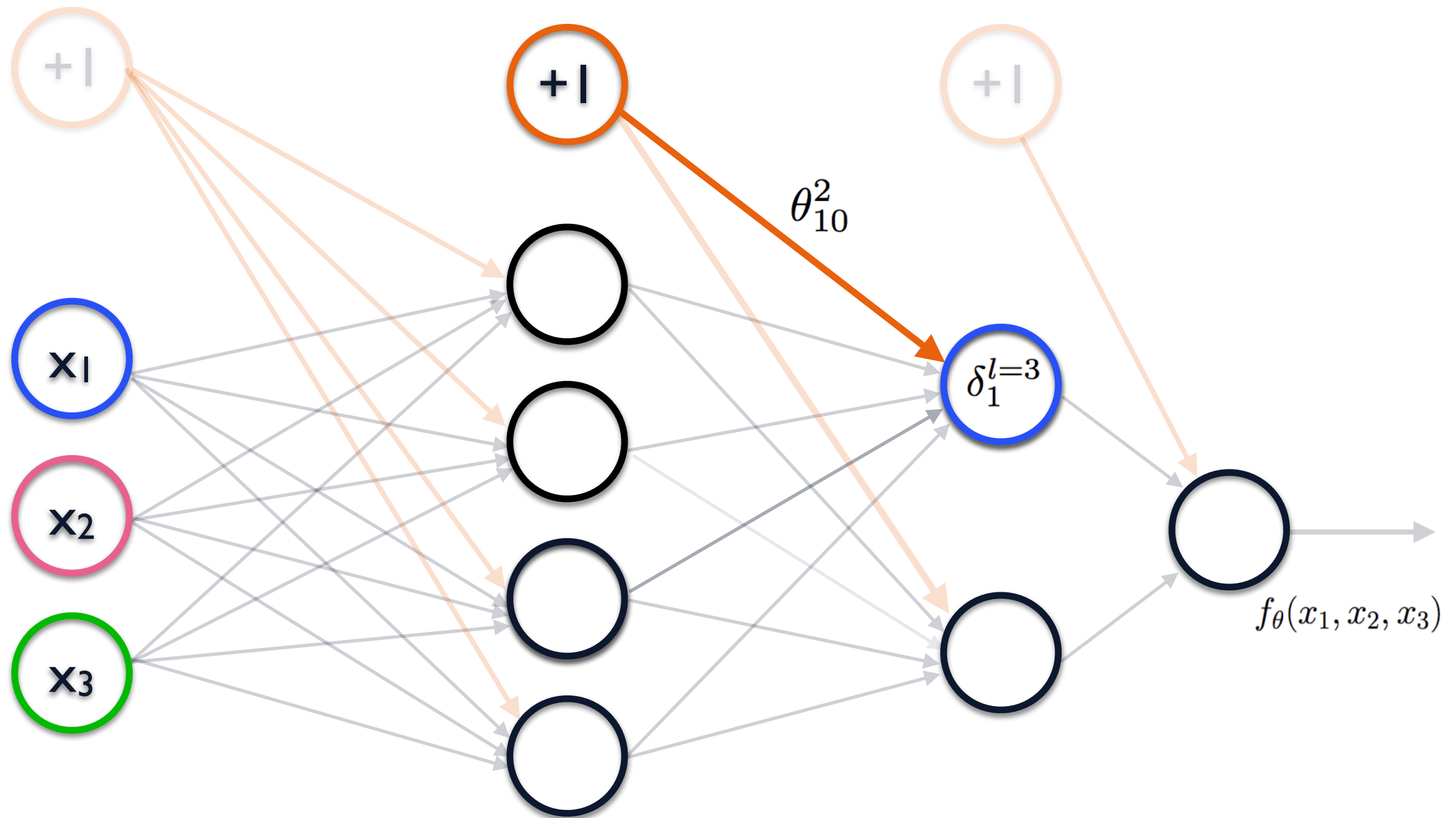
It is not necessary to compute a δ value for them!



$$\frac{\partial J}{\partial \theta_{10}^{l=2}} = \overset{?}{a_0^{l=2}} \delta_1^{l=3}$$

How to update the weights connected to a bias neuron?

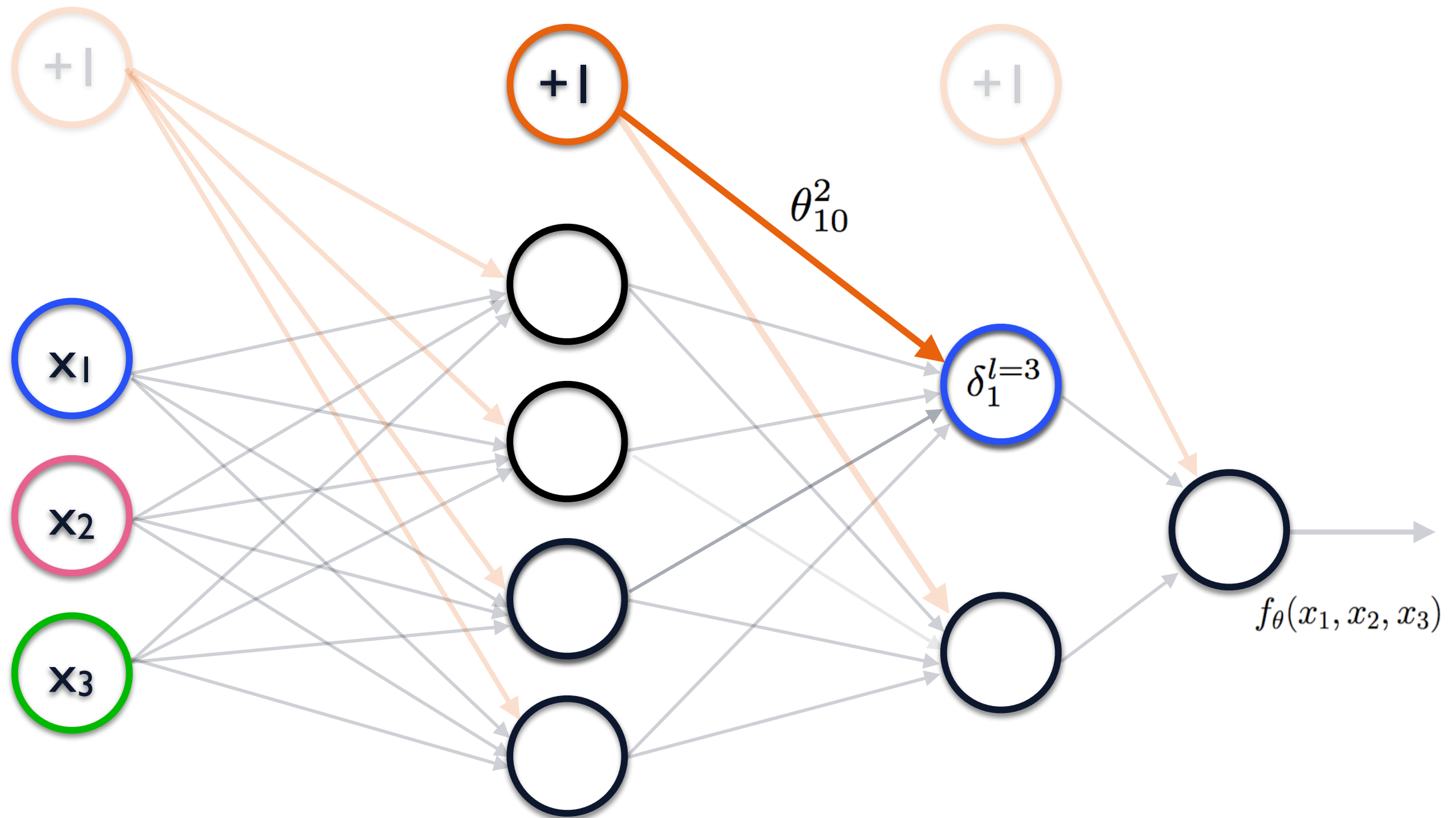
Gradient of a weight, based on network's error after a single training example is shown to the network



How to update the weights connected to a bias neuron?

$$\frac{\partial J}{\partial \theta_{10}^{l=2}} = 1 \delta_1^{l=3}$$

Gradient of a weight, based on network's error after a single training example is shown to the network



$$\frac{\partial J}{\partial \theta_{10}^{l=2}} = \delta_1^{l=3}$$

How to update the weights connected to a bias neuron?

Gradient of a weight, based on network's error after a single training example is shown to the network

Backpropagation Algorithm

1. Initialize all weights of the network with non-zero random values

2. For each instance $(\mathbf{x}, \mathbf{y})$ in the training set:

1. Propagate the instance through the network, computing its output/prediction $\mathbf{f}_\theta(\mathbf{x})$

2. Compute the δ values for all neurons in the output layer

$$\delta_1^{l=L} = (f_\theta(x) - y)$$

3. Compute the δ values for all neurons in the hidden layers:

(not needed for *bias* neurons)

$$\delta_i^{l=k} = \left(\sum_{j=1}^N \theta_{ij}^{l=k} \delta_j^{l=k+1} \right) (a_i^{l=k}) (1 - a_i^{l=k})$$

4. Computes the gradients of all weights of the network:

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \delta_i^{l=k+1} \quad \text{where } a_j^{l=k} = 1 \text{ for bias neurons}$$

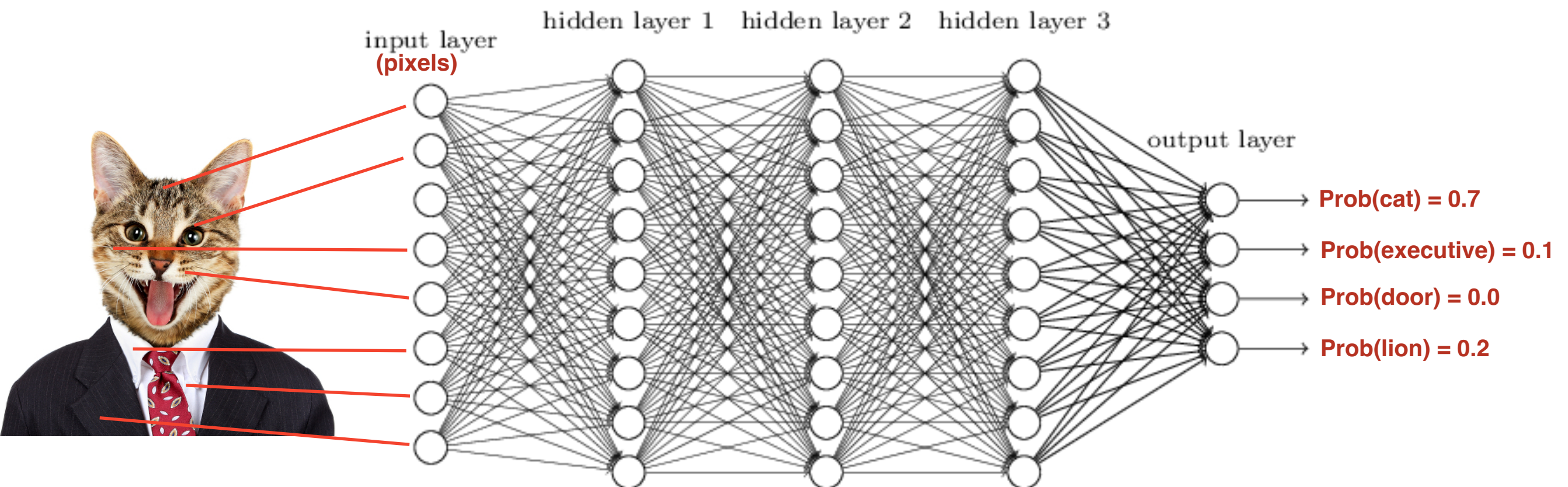
5. Adjust all weights of the network according to the gradient descent update rule:

$$\theta_{ij}^l := \theta_{ij}^l - \alpha \frac{\partial J}{\partial \theta_{ij}^l}$$

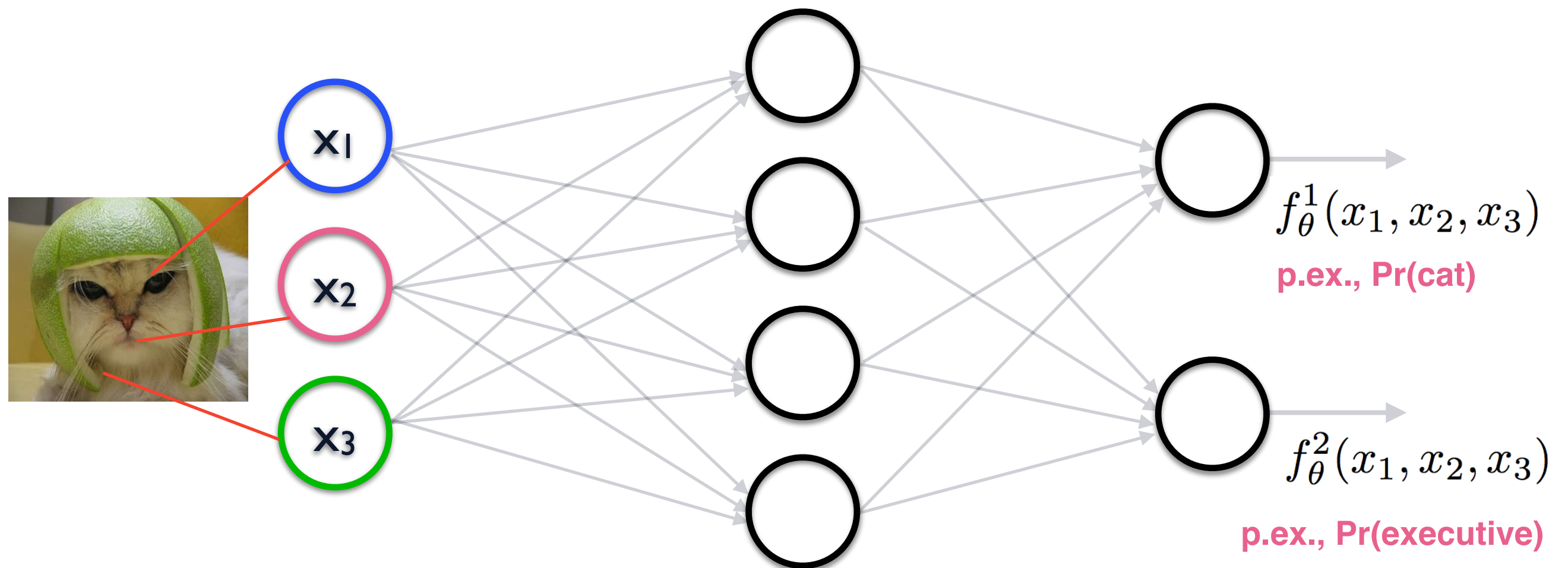
3. Evaluate the performance, J , of the network

1. If sufficiently good (or if improvement is too small), then stop

Networks with Multiple Outputs



Networks with Multiple Outputs

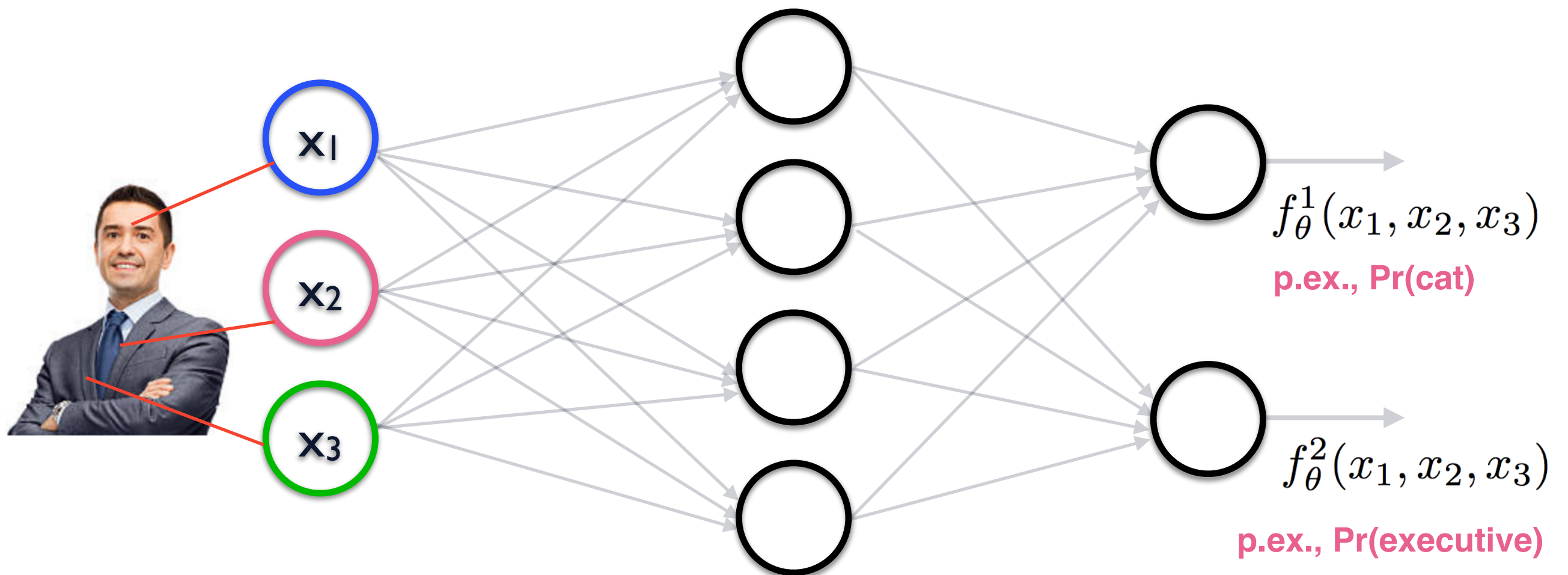


Input x :



Expected outputs: $y_1 = 1.0$
 $y_2 = 0.0$

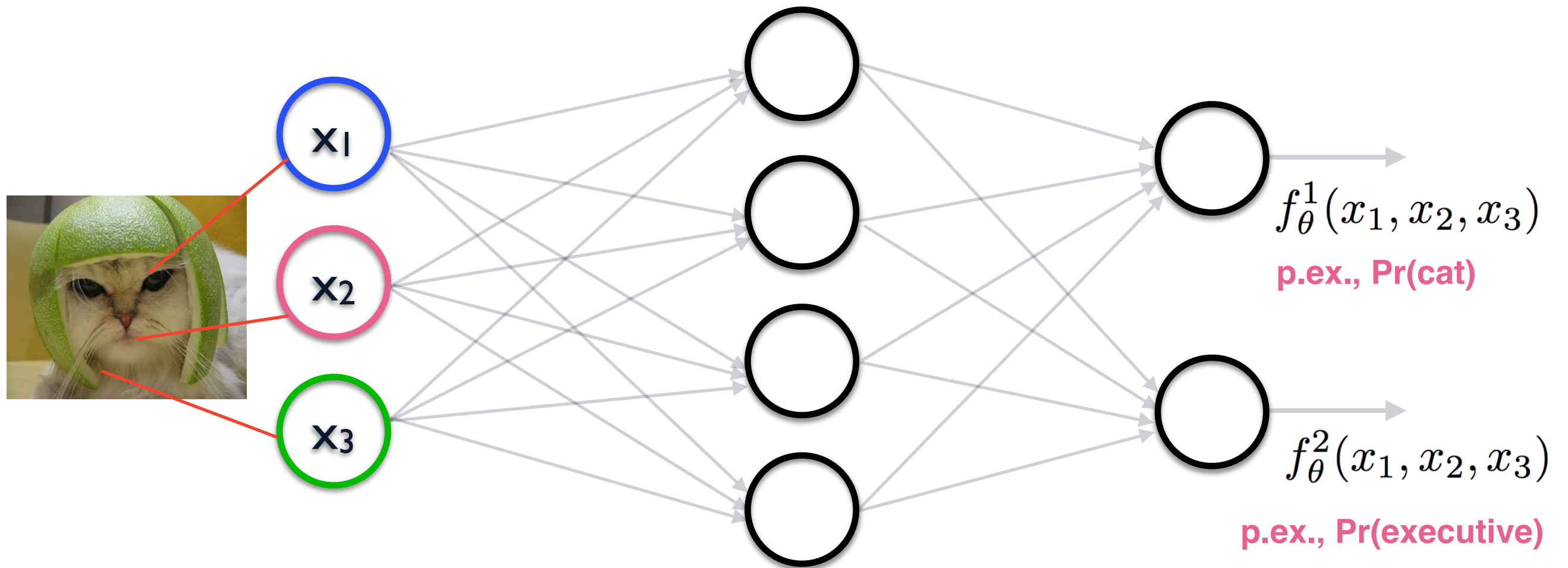
Networks with Multiple Outputs



Input $\mathbf{x}$:

Expected outputs: $y_1 = 0.0$
 $y_2 = 1.0$

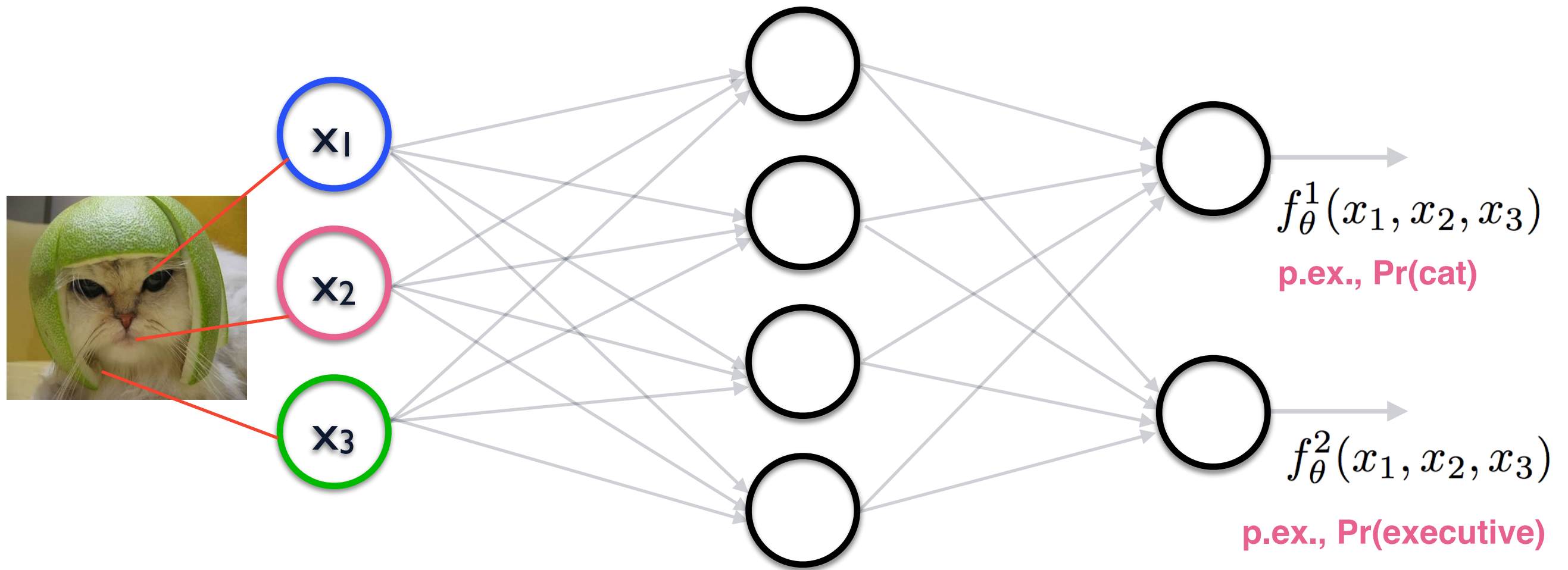
Networks with Multiple Outputs



Expected outputs: $y_1 = 0.0$
 $y_2 = 1.0$

First output's error: $-y_1 \left(\log f_{\theta}^1(x) \right) - (1 - y_1) \left(\log(1 - f_{\theta}^1(x)) \right)$

Networks with Multiple Outputs

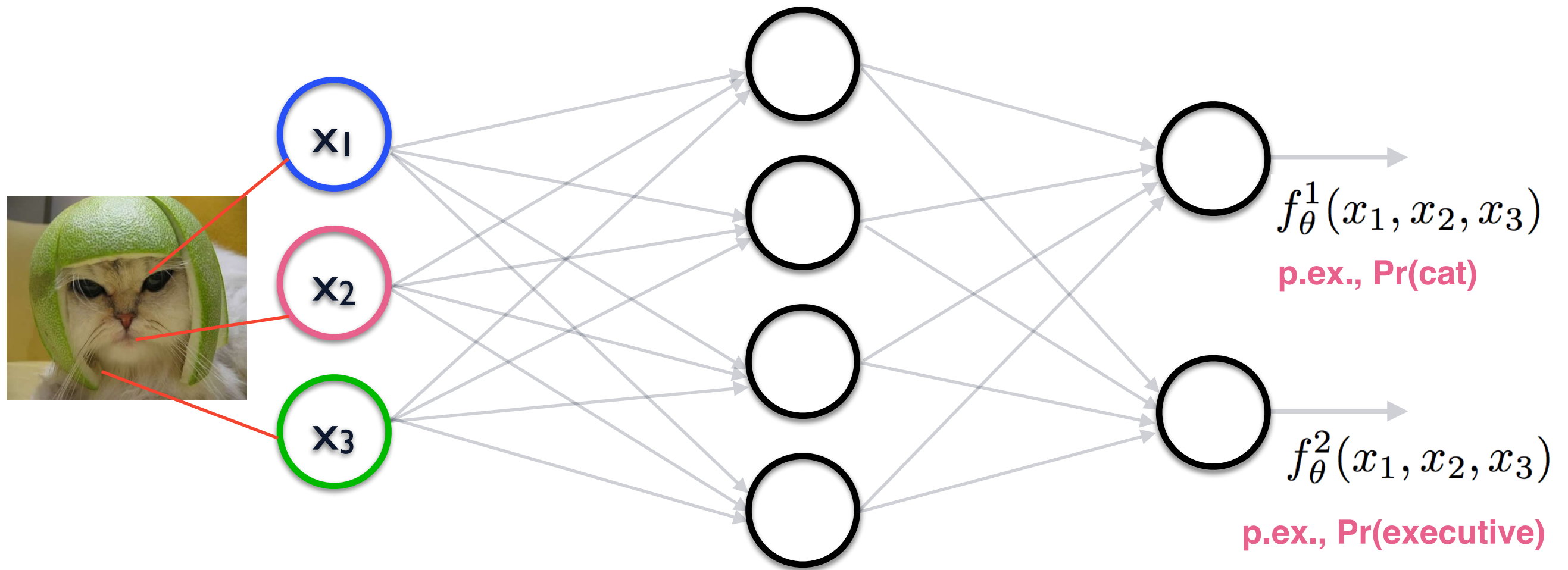


Expected outputs: $y_1 = 0.0$
 $y_2 = 1.0$

First output's error: $-y_1 \left(\log(f_{\theta}^1(x)) \right) - (1 - y_1) \left(\log(1 - f_{\theta}^1(x)) \right)$

Second output's error: $-y_2 \left(\log(f_{\theta}^2(x)) \right) - (1 - y_2) \left(\log(1 - f_{\theta}^2(x)) \right)$

Networks with Multiple Outputs

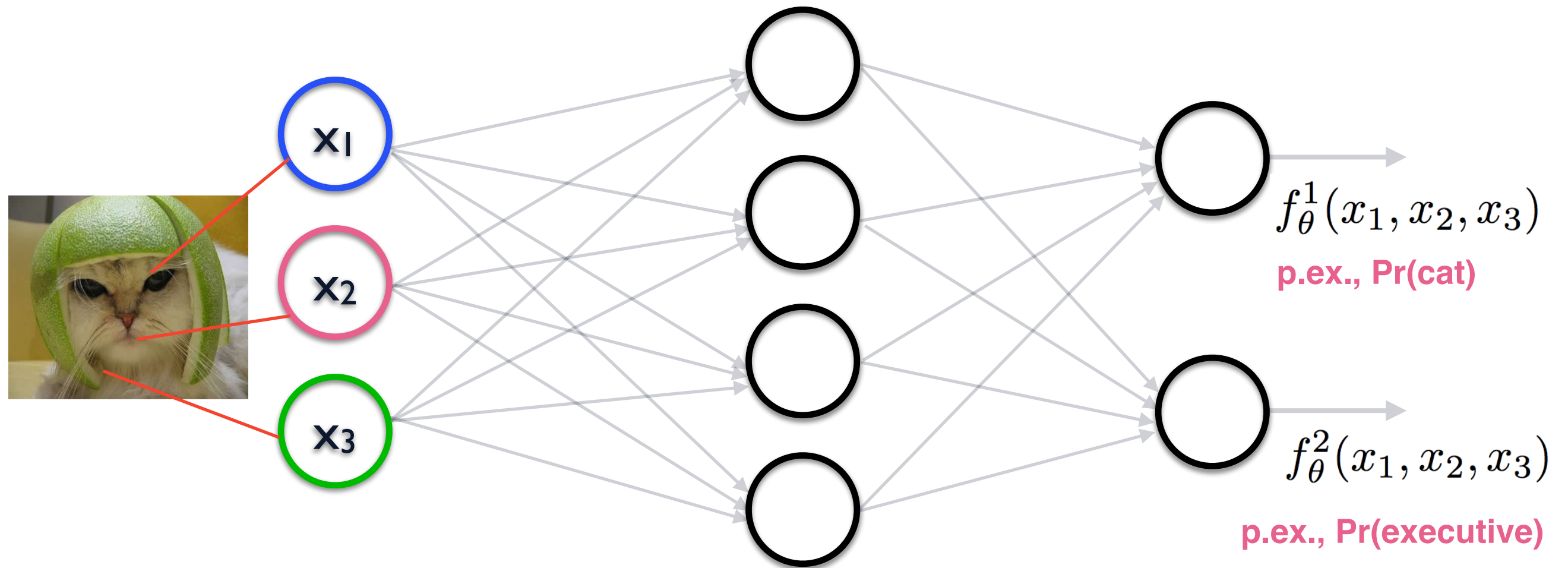


Expected outputs: $y_1 = 0.0$
 $y_2 = 1.0$

Total error
for the current
instance:

$$\begin{aligned} & -y_1 \left(\log(f_{\theta}^1(x)) \right) - (1 - y_1) \left(\log(1 - f_{\theta}^1(x)) \right) \\ & + \\ & -y_2 \left(\log(f_{\theta}^2(x)) \right) - (1 - y_2) \left(\log(1 - f_{\theta}^2(x)) \right) \end{aligned}$$

Networks with Multiple Outputs

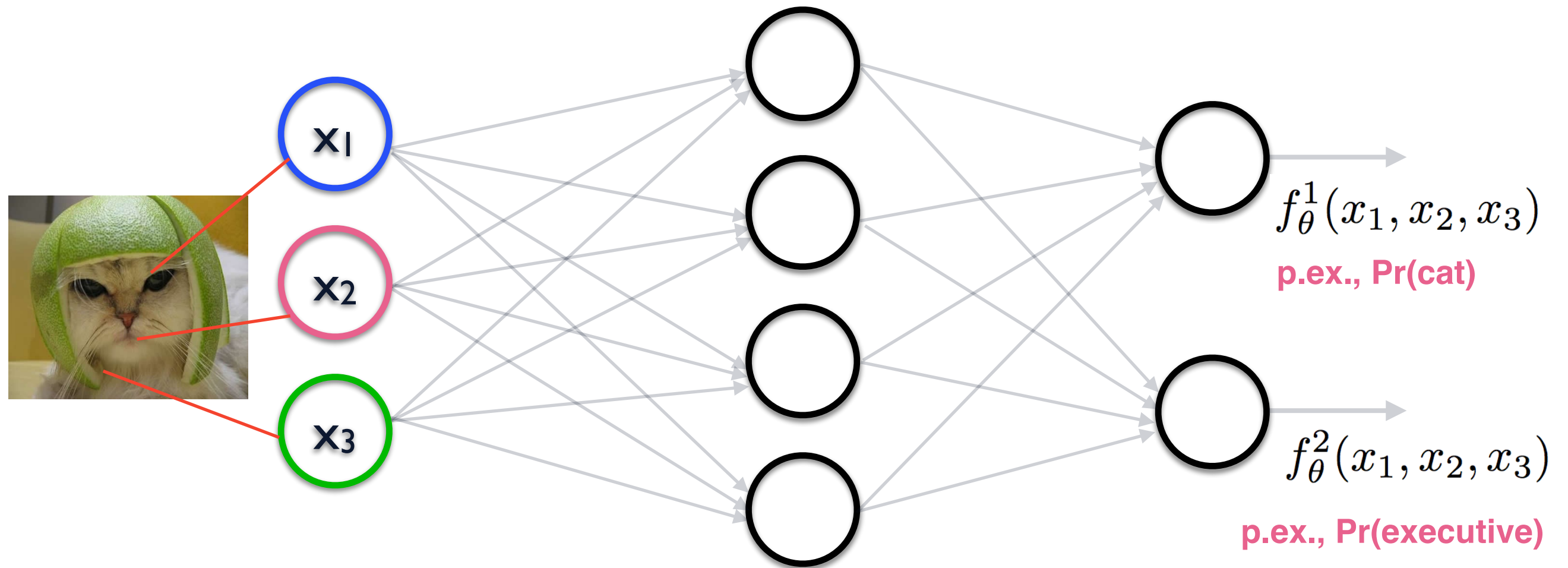


Now we need to sum these errors over all training instances

$$\sum_{k=1}^K -y_k \left(\log(f_{\theta}^k(x)) \right) - (1 - y_k) \left(\log(1 - f_{\theta}^k(x)) \right)$$

(where K is the number of outputs of the network)

Networks with Multiple Outputs

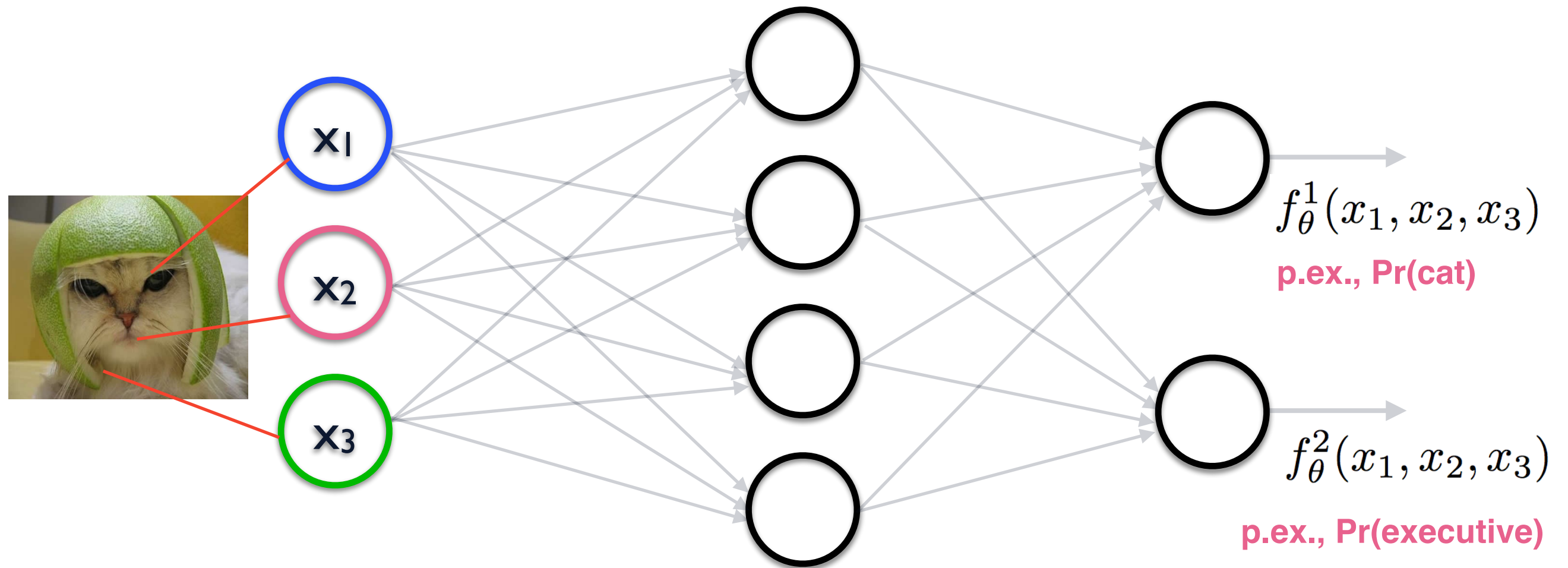


Now we need to sum these errors over all training instances

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{(i)} \left(\log(f_{\theta}^k(x^{(i)})) \right) - (1 - y_k^{(i)}) \left(\log(1 - f_{\theta}^k(x^{(i)})) \right)$$

(where K is the number of outputs of the network)

Networks with Multiple Outputs

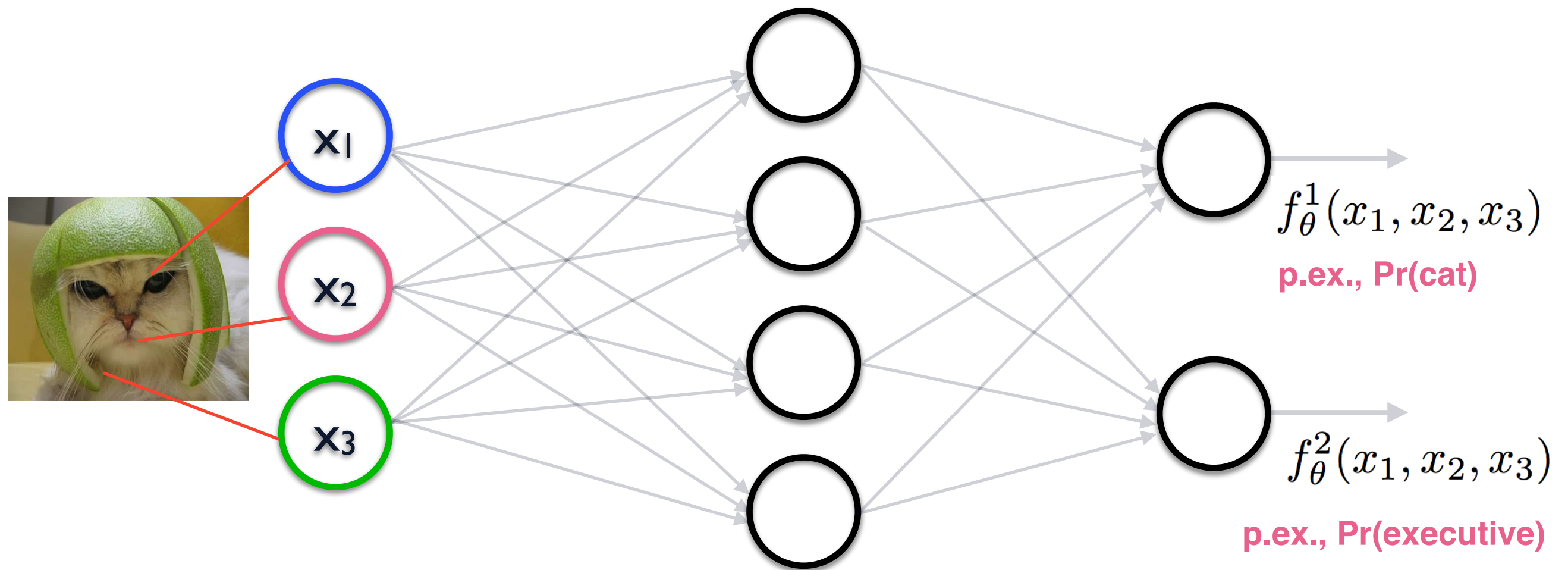


Cost function for networks with multiple outputs

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{(i)} \left(\log(f_{\theta}^k(x^{(i)})) \right) - (1 - y_k^{(i)}) \left(\log(1 - f_{\theta}^k(x^{(i)})) \right)$$

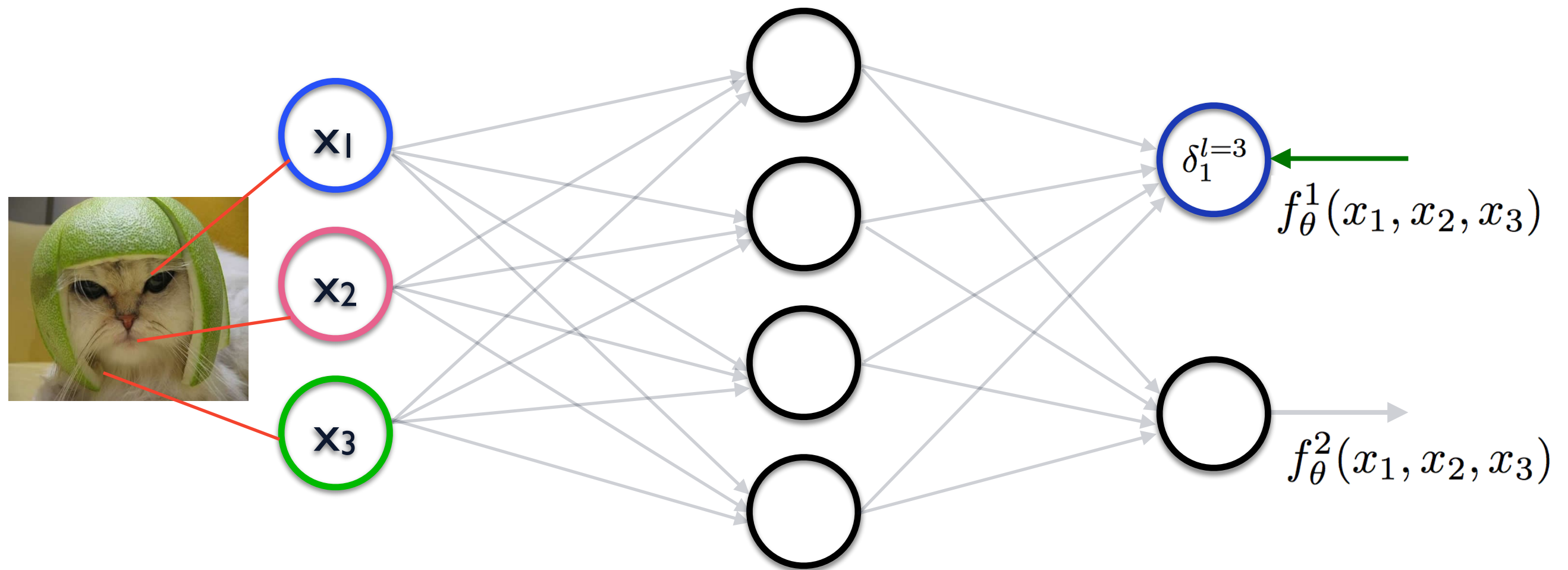
(where K is the number of outputs of the network)

Networks with Multiple Outputs



How does this modification affect the process of computing gradients?

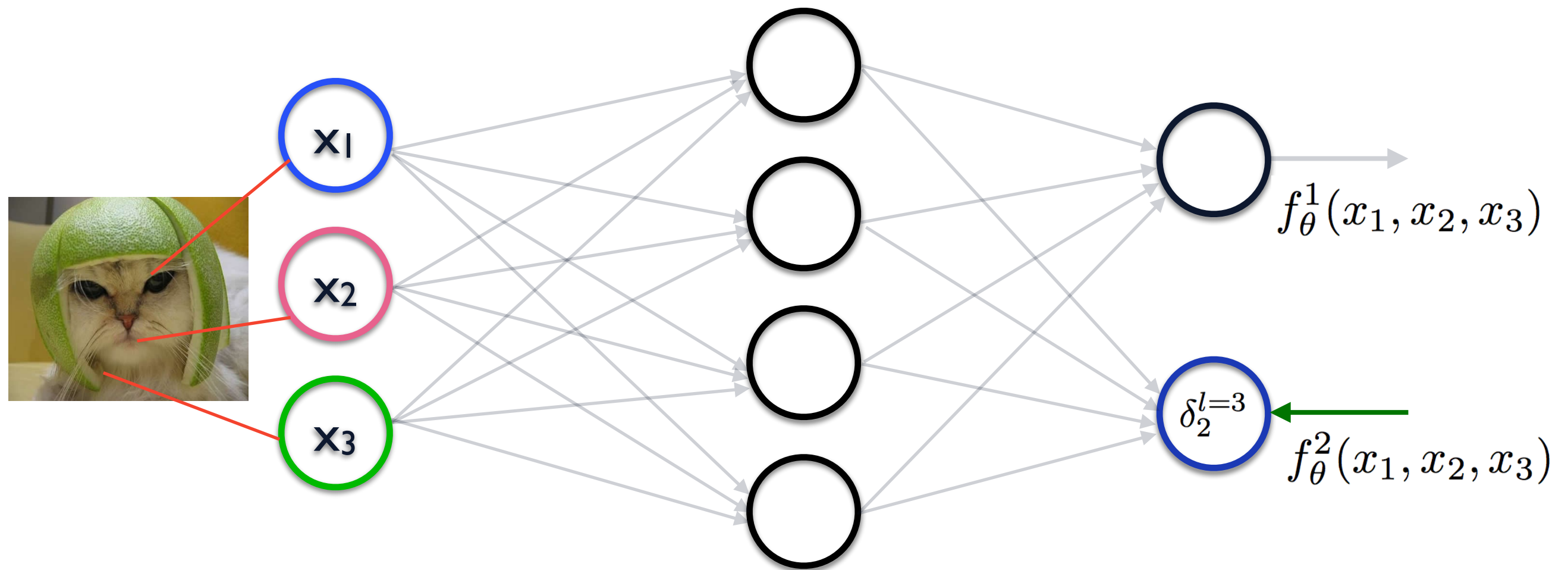
Networks with Multiple Outputs



$$\delta_1^{l=3} = \left(f_{\theta}^1(x) - y_1 \right)$$

δ of an output neuron $\rightarrow$ quantifies the error in that neuron's output

Networks with Multiple Outputs

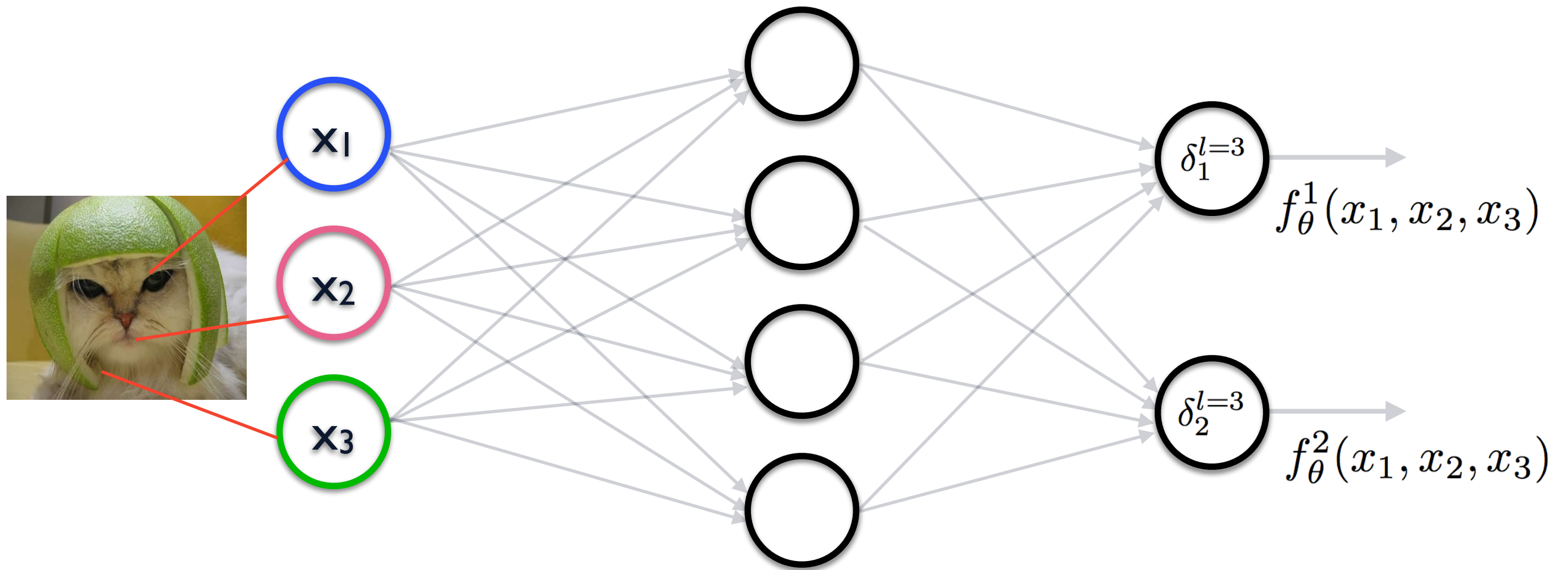


$$\delta_1^{l=3} = \left(f_{\theta}^1(x) - y_1 \right)$$

$$\delta_2^{l=3} = \left(f_{\theta}^2(x) - y_2 \right)$$

δ of an output neuron $\rightarrow$ quantifies the error in that neuron's output

Networks with Multiple Outputs



$$\delta_1^{l=3} = \left(f_{\theta}^1(x) - y_1 \right)$$

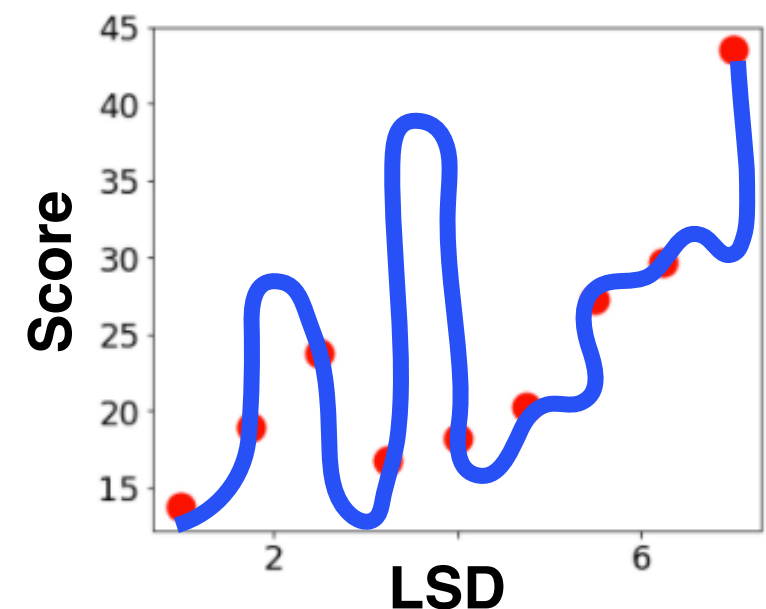
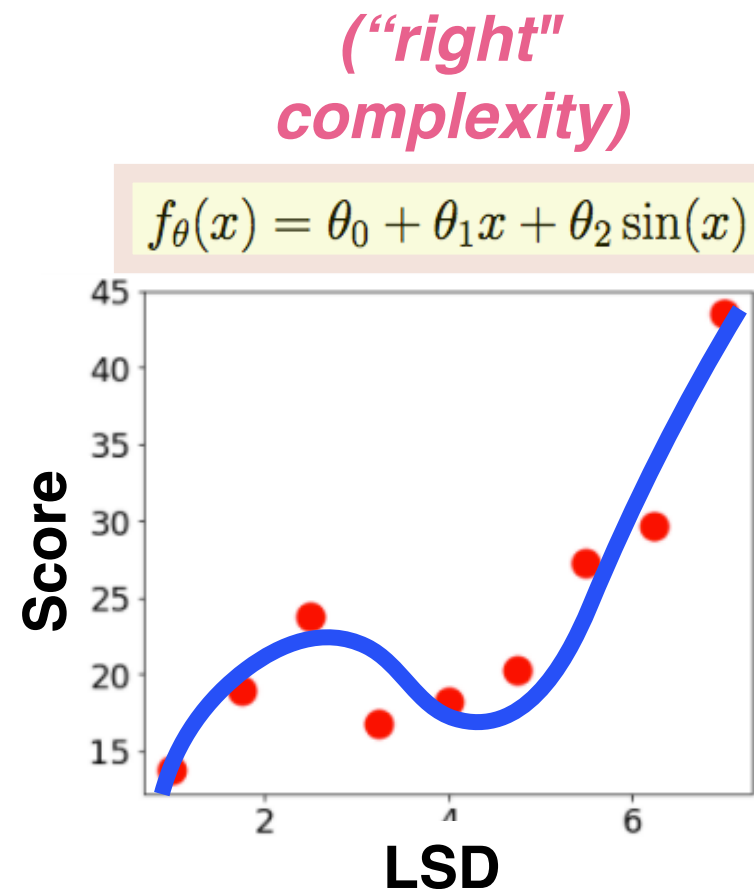
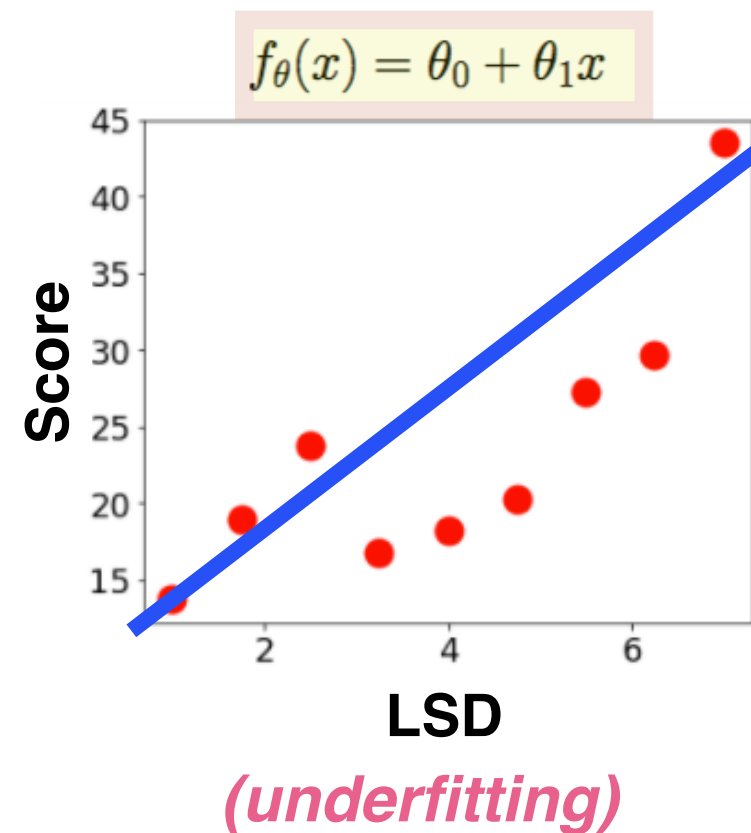
$$\delta_2^{l=3} = \left(f_{\theta}^2(x) - y_2 \right)$$

**All other calculations
(i.e., of all other δ values and gradients)
remain the same!**

Regularization

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$



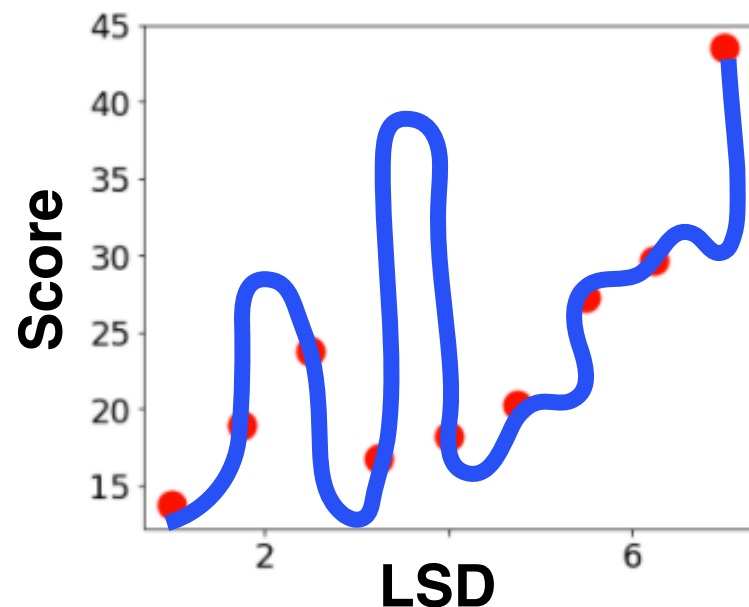
$$\begin{aligned} f_{\theta}(x) = & \theta_0 + \theta_1 x + \theta_2 \sin(x) + \theta_3 \cos(x) \\ & + \theta_3 x^7 + \theta_4 \exp(x) + \theta_5 \tan(x^3) + \dots \end{aligned}$$

(overfitting)

Regularization

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$



$$\begin{aligned} f_{\theta}(x) &= \theta_0 + \theta_1 x + \theta_2 \sin(x) + \theta_3 \cos(x) \\ &+ \theta_3 x^7 + \theta_4 \exp(x) + \theta_5 \tan(x^3) + \dots \end{aligned}$$

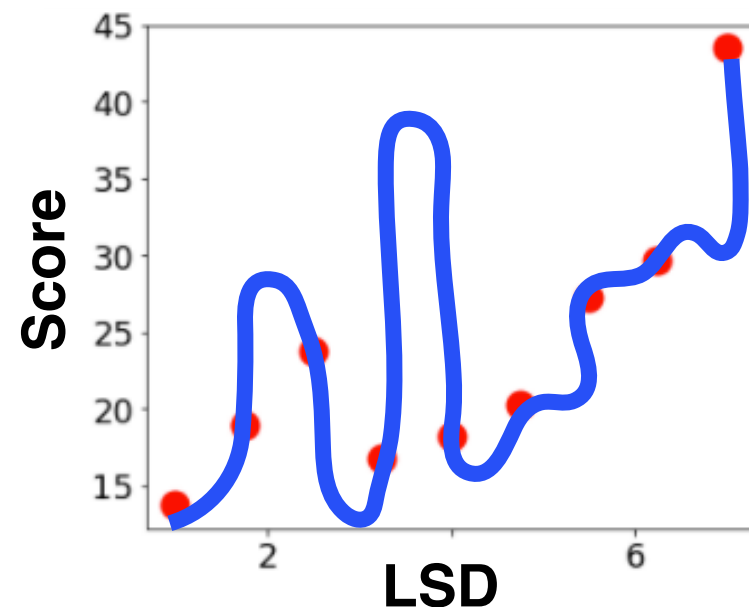
(overfitting)

- **Too many features**
- **Hypothesis is excessively complex to explain the patterns in the training data**
- **It is expressive enough that it can “memorize” the training data**
- **Correct predictions on all training instances**
- **But fails if evaluated on new data/examples**

Regularization

Linear Regression: find values of θ that minimize the cost function J

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$



$$\begin{aligned} f_{\theta}(x) &= \theta_0 + \theta_1 x + \theta_2 \sin(x) + \theta_3 \cos(x) \\ &+ \theta_3 x^7 + \theta_4 \exp(x) + \theta_5 \tan(x^3) + \dots \end{aligned}$$

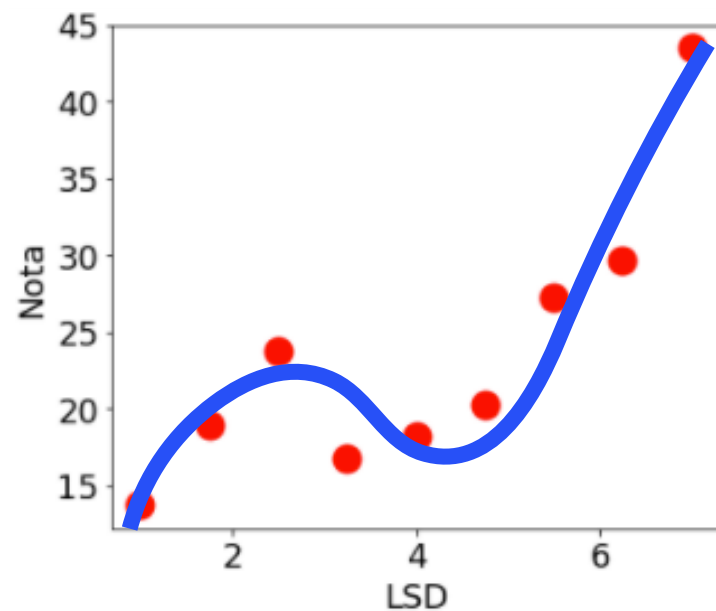
(overfitting)

- Too many features
- Hypothesis is excessively complex to explain the patterns in the training data
- It is expressive enough that it can “memorize” the training data
- Correct predictions on all training instances
- But fails if evaluated on new data/examples

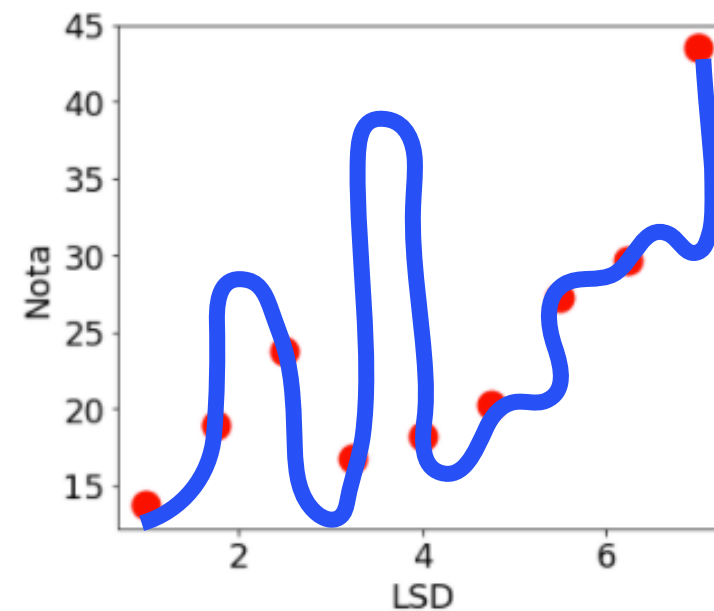
**This problem also affects neuron networks
(with too many neurons and/or layers)**

Regularization

- Regularization
 - keeps all the features, but forces the model to use as few as possible
 - forces the magnitude of the weights θ be as small as possible



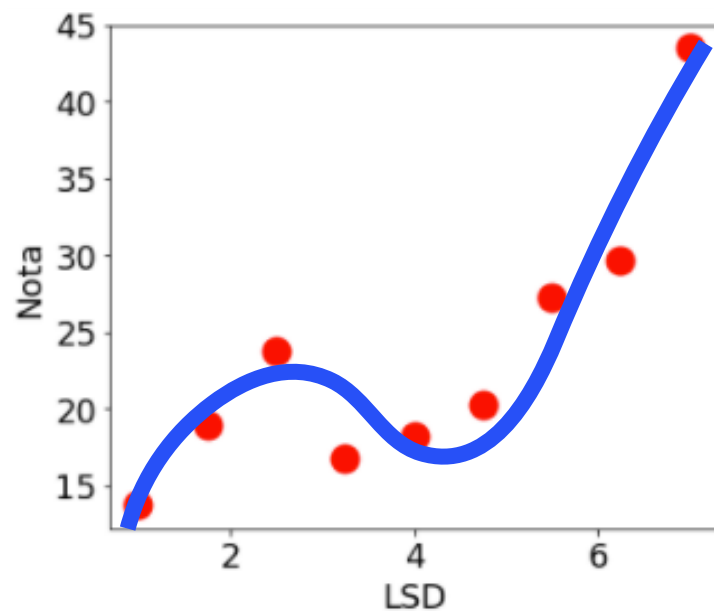
$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 \sin(x)$$



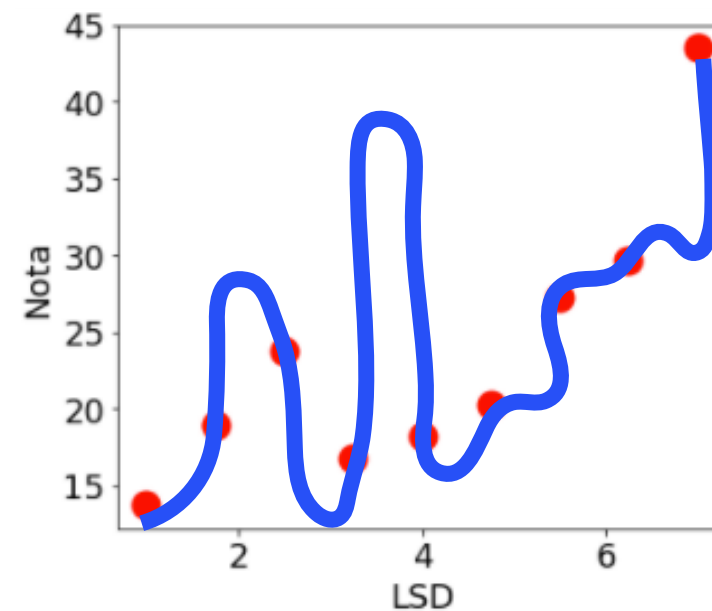
$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 \sin(x) + \theta_3 x^7 + \theta_4 \tan(x^3)$$

Regularization

- Regularization
 - keeps all the features, but forces the model to use as few as possible
 - forces the magnitude of the weights θ be as small as possible

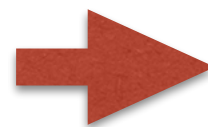


$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 \sin(x)$$



$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 \sin(x) + \theta_3 x^7 + \theta_4 \tan(x^3)$$

Cost function to find parameters/weights that minimize the prediction error

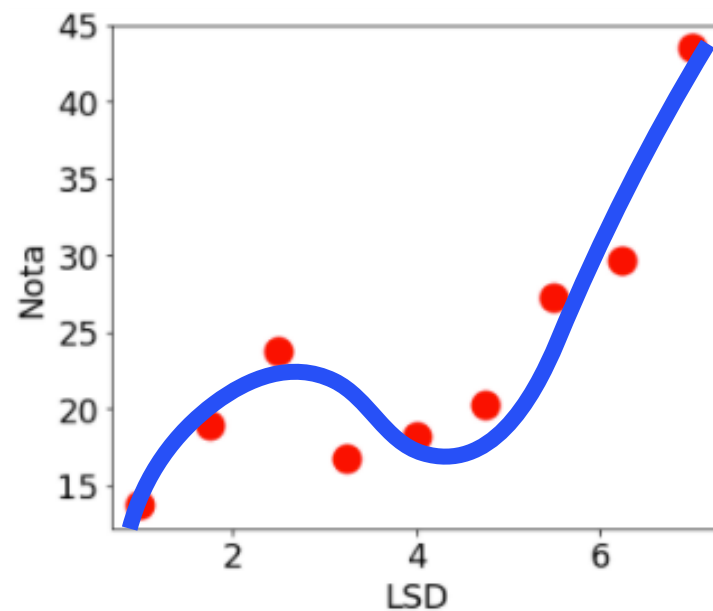


$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$

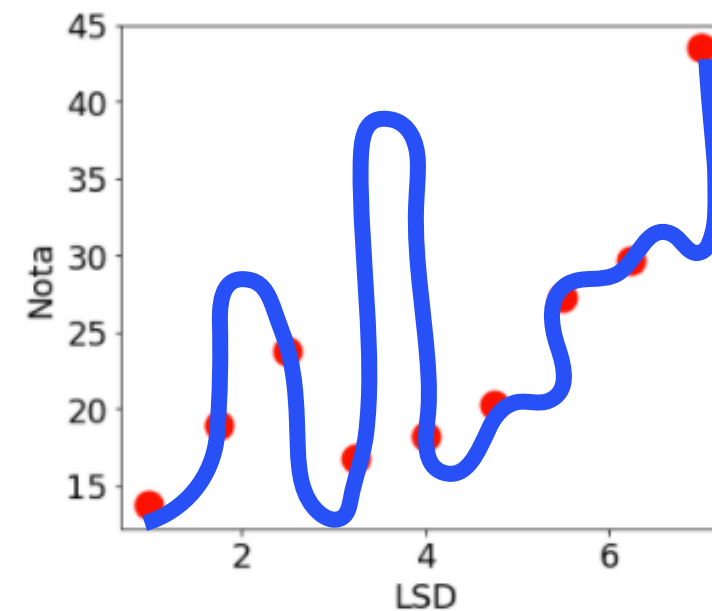
What if we penalize using features x^7 and $\tan(x^3)$?

Regularization

- Regularization
 - keeps all the features, but forces the model to use as few as possible
 - forces the magnitude of the weights θ be as small as possible



$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 \sin(x)$$



$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 \sin(x) + \theta_3 x^7 + \theta_4 \tan(x^3)$$

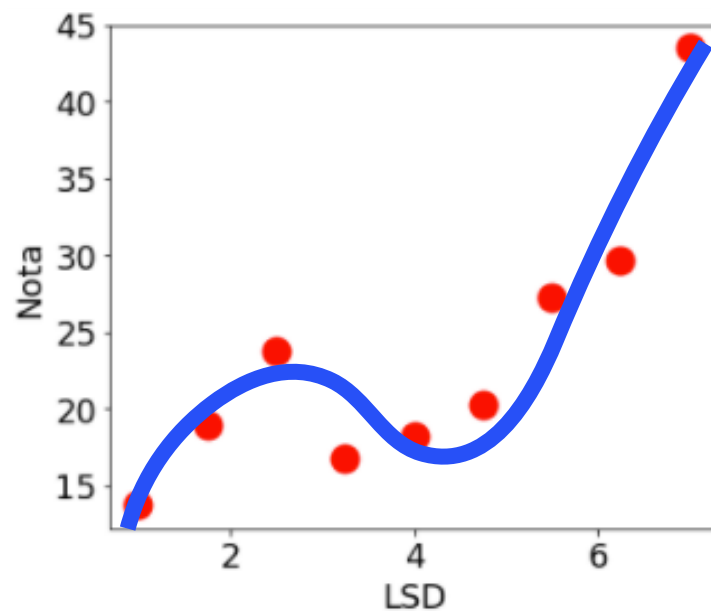
$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2 + 100000 \times (\theta_3)^2 + 100000 \times (\theta_4)^2$$

What if we penalize using features x^7 and $\tan(x^3)$?

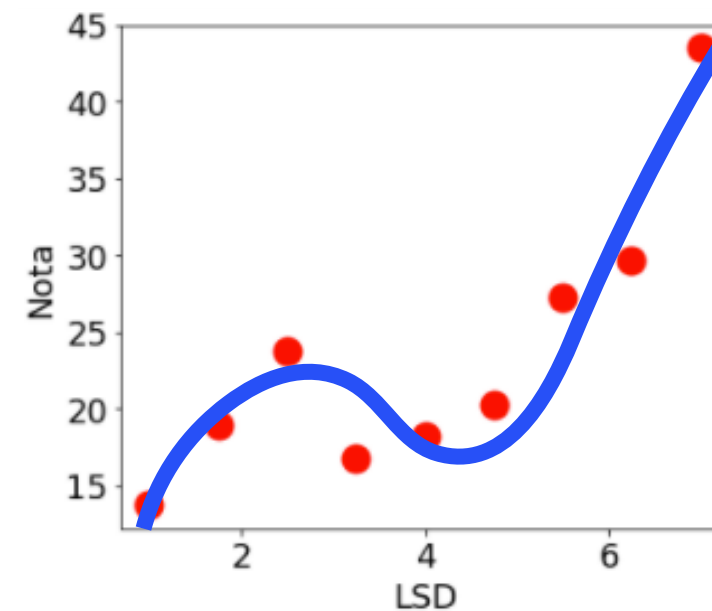
Will force $\theta_3 \rightarrow 0$ and $\theta_4 \rightarrow 0$

Regularization

- Regularization
 - keeps all the features, but forces the model to use as few as possible
 - forces the magnitude of the weights θ be as small as possible



$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 \sin(x)$$



$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 \sin(x) + \theta_3 x^7 + \theta_4 \tan(x^3)$$

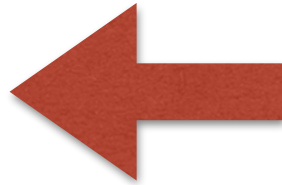
$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2 + 100000 \times (\theta_3)^2 + 100000 \times (\theta_4)^2$$

What if we penalize using features x^7 and $\tan(x^3)$?

Will force $\theta_3 \rightarrow 0$ and $\theta_4 \rightarrow 0$

Regularization

- Regularization
 - keeps all the features, but forces the model to use as few as possible
 - forces the magnitude of the weights θ be as small as possible

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^K (\theta_j)^2$$


But in practice, we don't know which features to penalize

So we penalize all of them!

- Algorithm will use weights only if their corresponding features help minimize the prediction error

Regularization

- Regularization
 - keeps all the features, but forces the model to use as few as possible
 - forces the magnitude of the weights θ be as small as possible

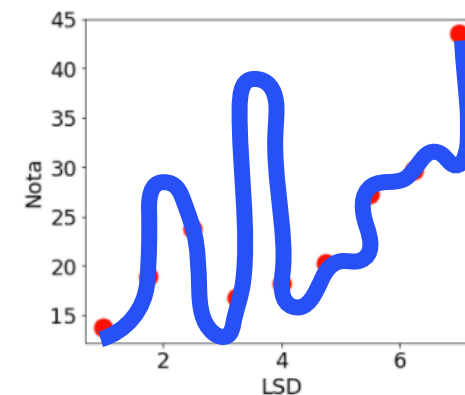
$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^K (\theta_j)^2$$

What happens if $\lambda=0$?

Does not penalize using any of the features
= standard Linear Regression

Might **overfit**

Excessively complex model → correct prediction for training data,
but incorrect ones on test data



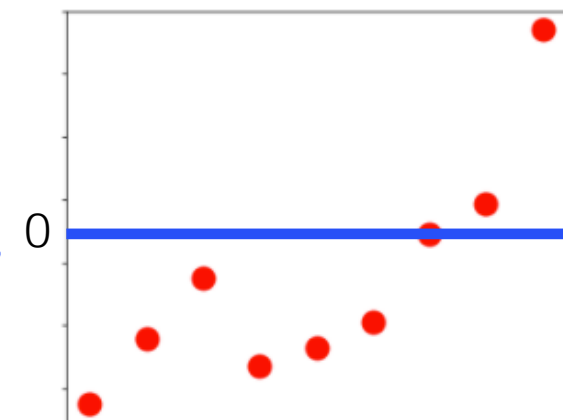
What happens if $\lambda=99999999$?

Extremely high penalty for using any weights θ that are not zero

The only way to minimize J is to force all weights $\theta \rightarrow 0$

Might **underfit**

Excessively simple model → doesn't even make correct predictions
on the training data



Regularized Linear Regression

Demo

cscheid.net

Regularization

- Regularized linear regression

$$J(\theta_1, \dots, \theta_K) = \frac{1}{2n} \left(\sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^K (\theta_j)^2 \right)$$

(last term should not add/include the *bias* term)

$$\frac{\partial}{\partial \theta_j} J(\theta_1, \dots, \theta_K) = \frac{1}{n} \sum_{i=1}^n \left(f_{\theta}(x^{(i)}) - y^{(i)} \right) \phi_j(x^{(i)}) + \frac{\lambda}{n} \theta_j$$

(last term should not be used when updating the *bias* weight)

- Regularized logistic regression

$$J(\theta_1, \dots, \theta_K) = \left(\frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1 - y^{(i)}) \left(\log(1 - f_{\theta}(x^{(i)})) \right) \right) + \frac{\lambda}{2n} \sum_{j=1}^K (\theta_j)^2$$

(last term should not add/include the *bias* term)

$$\frac{\partial}{\partial \theta_j} J(\theta_1, \dots, \theta_K) = \frac{1}{n} \sum_{i=1}^n \left(f_{\theta}(x^{(i)}) - y^{(i)} \right) \phi_j(x^{(i)}) + \frac{\lambda}{n} \theta_j$$

(last term should not be used when updating the *bias* weight)

- Regularized neural networks

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{(i)} \left(\log(f_{\theta}^k(x^{(i)})) \right) - (1 - y_k^{(i)}) \left(\log(1 - f_{\theta}^k(x^{(i)})) \right) + \frac{\lambda}{2n} \sum_l \sum_i \sum_j (\theta_{ij}^l)^2$$

(last term should not add/include the *bias* terms)

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \delta_i^{l=k+1} + \lambda \theta_{ij}^{l=k}$$

(last term should not be used when updating the *bias* weights)



Machine Learning

CMPSCI 589

Bruno C. da Silva

bsilva@cs.umass.edu

Neural Networks (pt. 2)

UMassAmherst
College of Information
& Computer Sciences



This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/)

